応用情報技術者試験 科目A 知識要点（印刷用）

過去問8回分（令和4年度春期〜令和7年度秋期）の論点を中分類ごとに整理。計算だけの論点は除外。

01 基礎理論

論理式・カルノー図・論理回路

① 記号と6つの基本演算

記号：A·B＝AND（論理積）、A＋B＝OR（論理和）、$\bar{A}$＝NOT（否定）

A	B	AND	OR	NAND	NOR（Not OR、ORの否定）	XOR	XNOR（一致）
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

（表はコードで確認済み）

- **NAND（Not AND）**：ANDの結果を反転したもの。両方1のときだけ0。NANDだけを組み合わせれば、どんな回路も作れます。
- **NOR（Not OR）**：ORの結果を反転したもの。両方0のときだけ1。
- **XOR（eXclusive OR、排他的論理和）**：2つの入力が**異なる**とき1。
- **XNOR（一致回路）**：2つの入力が**同じ**とき1。XORの反転です。過去問では「入力XとYが同じときだけ1を出す回路」として出ました。

② ド・モルガンの法則（よく使います）

- ¬(A·B) ＝ $\bar{A}$＋$\bar{B}$（「AかつB」の否定は「Aでない、またはBでない」）
- ¬(A＋B) ＝ $\bar{A}$·$\bar{B}$（「AまたはB」の否定は「Aでなく、かつBでない」）
- 覚え方：上の線（否定）を切り離すと、·と＋が入れ替わります。

③ 多数決回路（3入力のうち2つ以上が1なら1）

- 式は A·B＋B·C＋C·A です（コードで確認済み）。
- 「どの2つを組み合わせても、両方1なら1」という意味になります。

④ カルノー図の解き方

1. 1のマスを、**1・2・4・8個**長方形でまとめます。大きくまとめるほど式が短くなります。
2. **左右の端どうし、上下の端どうしも隣**として扱います（図を丸めて筒にしたイメージです）。
3. 1つの長方形の中で、**0と1の両方が出てくる変数は消します**。値が変わらない変数だけを残します。
4. 長方形ごとの式を＋（OR）でつなげば完成です。

例：B＝1かつD＝1のマスが4つある場合、その中ではAとCが0にも1にもなるので消えます。残るのはBDです（コードで確認済み）。 ****：上の段の左右両端の2マスで $A\bar{B}\bar{D}$、中央の4マスで BD。答えは $A\bar{B}\bar{D}$＋BD でした。

検算のしかた：選択肢の式に、1のマスを1つ入れてみます。結果が0になる選択肢は間違いなので消せます。

⑤ RSフリップフロップ

- 1ビットを記憶する回路です。S（Set）＝1にすると、出力が1になります。R（Reset）＝1にすると、出力が0になります。
- どちらも0のときは、直前の値をそのまま保ちます。これが「記憶」です。

CRC・パリティ（誤り検出）

① CRC（Cyclic Redundancy Check、巡回冗長検査）

- データを送るときに、途中で壊れていないか（誤りがないか）を**検出する**方式です。
- 手順
  1. **送信側**：データを**生成多項式**で割り、その**余り**を検査用のデータ（CRC値）としてデータの後ろに付けて送ります。
     - 生成多項式とは、送信側と受信側であらかじめ決めておく「割る数」のことです。
  2. **受信側**：受け取った全体（データ＋CRC値）を、**送信側と同じ生成多項式**で割ります。余りが0なら「誤りなし」と判断します。
- 特徴：連続して起きる誤り（バースト誤り）の検出に強い方式です。誤りを**検出するだけで、訂正はしません。**

選択肢の見分け方

選択肢	正誤	理由
送信側で、生成多項式を使って検査用データを作り、データに付けて送る	○	
検査用のデータは1ビットの値	×	1ビットなのはパリティ。CRCは複数ビット
受信側では、付いてきた**検査用データで**データを割る	×	割るのは生成多項式
送信側と受信側で**異なる**生成多項式を使う	×	**同じ**生成多項式を使う

② パリティ

- **パリティビット**：データの中の1の個数が偶数（または奇数）になるように付け加える、1ビットの検査用データです。
  - 例：偶数パリティで、データ1011（1が3個）なら、パリティ1を付けて1の個数を4個（偶数）にします。
- **垂直・水平パリティ**：データを格子状に並べ、行ごとと列ごとにパリティを付ける方式です。1ビットの誤りなら、どの行とどの列のパリティが合わないかで位置が分かるので、**1ビットまで訂正できます。**

③ 似ているものとの区別

方式	検出	訂正
パリティ（1方向）	1ビット（奇数個）の誤り	できない
垂直・水平パリティ	1ビットの誤り	**1ビットまで**
ハミング符号	2ビットまでの誤り	**1ビットまで**
CRC	複数ビット、特に連続した誤り	**できない**

センサー（LiDAR・ミリ波・ドップラー）

① 自動車のセンサー4種類

センサー	何を出すか	天候の影響	得意なこと	できないこと
LiDAR（Light Detection And Ranging）、レーザーレーダー	**レーザー光**をパルス状（短く区切った光）に当てて、反射光を測る	受けやすい（雨や霧に弱い）	対象物の**方向・距離・形状**を細かく測る	建物の陰にある物は見えない（光は物を通り抜けないため）
ミリ波レーダー	**電波**（ミリ波帯の電磁波）を出して、反射波を測る	**受けにくい**	距離と相対速度を測る	色は分からない（信号機の色も判別できない）
超音波センサー	超音波（人に聞こえない高い音）を出して、反射を測る	受けやすい	数メートル以内の近距離（駐車のとき）	遠くは測れない（500mは誤り）
可視光カメラ	目に見える光で撮影する	受けやすい	**標識や信号機の色・文字の認識**	暗いときや悪天候に弱い

見分け方

- 「レーザー光」とあればLiDARです。「ミリ波」「電磁波」とあればミリ波レーダー、「超音波」とあれば超音波センサーです。

- 「色・文字を読む」のは、カメラだけです。

ひっかけ

- 「ミリ波レーダーで信号機の色を判別できる」は誤りです。電波では色は分かりません。
- 「レーザーレーダーは天候に強く、建物の後ろも見える」は誤りです。天候に弱く、物の陰も見えません。
- 「超音波センサーで500m以上測れる」は誤りです。測れるのは数m程度です。

② ドップラー効果

- 動いている物に当てて反射した波は、**近づく物からは周波数が高く**、**遠ざかる物からは低く**なります。この周波数のずれから**速度**が分かります。
- 例：救急車のサイレンが、近づくときは高く聞こえ、遠ざかると低く聞こえる現象です。
- 医療では、血液（赤血球）の動く速さを測ることで、**血流量**が分かります。
- ほかの選択肢の血中酸素飽和度・血糖値・体内水分量は、「動く速さ」ではないので、ドップラー効果では測れません。

チャタリング・リアルタイム応答

① 押された瞬間を見つける論理式

- 条件は次のとおりです。
  - スイッチは、放した状態で0、押した状態で1になります。
  - a＝今回読んだ値、b＝前回読んだ値です。
- 「**放した状態から押した状態に変わった**」とは、**前回が0（b＝0）で、今回が1（a＝1）**ということです。
- 「b＝0」は否定を付けて $\bar{b}$ と書きます。「かつ」は論理積・なので、答えは **$a \cdot \bar{b}$** です。

4つの組合せと意味

前回 b	今回 a	状態	式
0	1	**押された瞬間**	**$a \cdot \bar{b}$（正解）**
1	0	放された瞬間	$\bar{a} \cdot b$
1	1	押したまま	$a \cdot b$
0	0	放したまま	$\bar{a} \cdot \bar{b}$

- ＋（論理和、または）を使う選択肢（$a+\bar{b}$、$\bar{a}+b$）は誤りです。「変化した瞬間」は、2つの条件が**両方**そろったときだけなので、論理積（かつ）になります。

② チャタリング（chattering）

- スイッチを押した瞬間や放した瞬間に、金属の接点が細かく跳ねて、**0と1が短い時間に何度も入れ替わる**現象です。
- そのまま読むと、1回押しただけで何回も押されたと誤って判断してしまいます。
- 対策（チャタリング処理）：少し時間をおいて2回読み、**同じ値が続いたら確定**します。

③ リアルタイムな応答

- **リアルタイムシステム**：決められた時間内に処理を終えることが求められるシステムです。例：車のエアバッグ、ブレーキの制御。
- リアルタイムな応答とは「**定められた制限時間内に応答する**」ことです（正解）。単に「速い」という意味ではありません。
- 誤りの選択肢
  - 「OS（Operating System）を使わない」：リアルタイムOSを使うのが普通なので誤りです。
  - 「入力された順序を守る」：順番より時間内に終わることが大事なので誤りです。
  - 「入力時刻を記録する」：記録するだけでは間に合うことの保証にならないので誤りです。

浮動小数点の誤差（情報落ち・桁落ち）

- 情報落ち＝絶対値が大きく違う2数を足し引きすると、小さい方の下位桁が仮数部に入りきらず消える。防ぎ方は「小さい数どうしを先に足す」。

- 桁落ち＝ほぼ等しい2数を引くと有効桁が減る。丸め誤差＝桁数の制限による端数処理。打切り誤差＝無限の計算を途中でやめる。
- 見分け方：「大きい＋小さい」→情報落ち、「ほぼ同じ−ほぼ同じ」→桁落ち。

集合演算（和・積・差・補集合）

- A∩B＝両方に含まれる、A∪B＝どちらかに含まれる、A−B＝Aにあって B にない＝A∩¬B。
- 関係代数の差 R−S、積 R∩S＝R−(R−S)。和両立＝列の数と型がそろっていて和・差・積が取れる関係。
- 解き方：ベン図を描いて領域に色を塗り、選択肢の式も塗って一致するものを選ぶ。

ハミング符号

- 冗長ビットを付けて「1ビットの誤りを訂正、2ビットの誤りを検出」できる符号。例：データ4ビット＋検査3ビット＝7ビット。
- 解き方：問題の検査式（P1＝X1⊕X2⊕X4 など）に受信値を入れ、0にならない式の組合せから誤り位置を特定して反転する。

確率・統計（正規分布・期待値）

- 正規分布：平均μ±1σに約68%、±2σに約95%、±3σに約99.7%。平均を変えると山の位置、σを変えると山の幅が変わる。
- 期待値＝Σ(値×確率)。「少なくとも1回」＝1−（1回も起きない確率）。ロットの抜取検査は「不良が○個以下で合格」の確率を足し合わせる。

AI：機械学習の基本（教師あり・なし・強化）

- 教師あり学習＝正解ラベル付きデータで学ぶ（分類・回帰）。教師なし学習＝ラベルなしでまとまりを見つける（クラスタリング）。強化学習＝行動の結果の報酬が最大になる方策を学ぶ。
- ディープラーニング＝多層のニューラルネットワーク。特徴量を人が設計せず自動で学ぶ。行列演算が多いので並列計算に強い GPU（Graphics Processing Unit）で高速化する。
- 活性化関数＝ニューロンの出力を非線形に変える関数（ReLU（Rectified Linear Unit、0未満を0にする関数）、シグモイドなど）。

AI：過学習と交差検証

- 過学習（overfitting）＝訓練データには高精度だが、未知のデータ（検証データ）で精度が下がる状態。兆候：訓練誤差は下がり続け、検証誤差が途中から上がる。
- 対策：データを増やす、モデルを単純にする、正則化、早期終了（early stopping）、ドロップアウト。
- 交差検証（k分割交差検証）＝データをk個に分け、1個を検証用・残りを訓練用にする処理をk回繰り返して平均で評価する。目的は汎化性能の評価（過学習の検出）。

AI：2クラス分類の評価（ROC・混同行列）

- 混同行列：TP（真陽性）・FP（偽陽性＝陰性を陽性と誤判定）・FN（偽陰性）・TN（真陰性）。
- 再現率（recall）＝TP/(TP+FN)、適合率（precision）＝TP/(TP+FP)、正解率＝(TP+TN)/全体。
- ROC曲線（Receiver Operating Characteristic curve）＝横軸：偽陽性率 FP/(FP+TN)、縦軸：真陽性率 TP/(TP+FN)。左上に近いほど良い。AUC（Area Under the Curve、曲線の下の面積）は1に近いほど良く、0.5はランダムと同じ。

多変量解析（主成分分析・クラスタ分析など）

- 主成分分析＝多数の変数を、情報をなるべく失わずに少数の合成変数（主成分）にまとめる。
- クラスタ分析＝似たもの同士をグループに分ける（教師なし）。重回帰分析＝複数の説明変数から1つの目的変数を予測する。判別分析＝どのグループに属するかを判別する。

逆ポーランド記法（後置記法）

- 演算子を被演算子の後ろに書く。例：A+B×C → ABC（Activity-Based Costing、活動基準原価計算）×+、(A+B)×C → AB+C×。
- 計算はスタック：数は積み、演算子が来たら上の2つを取り出して計算し結果を積む。

XML・文字コード

- XML（eXtensible Markup Language、タグでデータを記述する言語）宣言の符号化宣言（encoding）は UTF-8（Unicodeの符号化方式の一つ） か UTF-16 なら省略できる（それ以外の文字コードは必須）。

再帰（階乗など）

- 再帰＝関数が自分自身を呼ぶ。終了条件（例：n=0 なら1）が必要。fact(n)=n×fact(n−1)。呼出しごとに戻り先をスタックに積む。

ビット演算・2進数の桁数

- マスク：x AND 1 で最下位ビットを取り出す、x OR m で特定のビットを1にする、x XOR（eXclusive OR、排他的論理和） m で特定のビットを反転する。
- 左シフト1ビット＝×2、右シフト1ビット＝÷2（切捨て）。2桁の2進数 x1x0 を入れ替えた値＝2×(x mod 2)＋int(x/2)。
- nビットの数どうしの掛け算は最大 2n ビット（a^2 は最大2nビット）。
- 255 の次を 0 に戻すような循環は「(n＋1) AND 255」（下位8ビットだけ残す）で書ける。

02 アルゴリズムとプログラミング

プログラム言語・データ形式

① 言語ごとの特徴

言語	見分けるポイント
Python	**字下げ（インデント）でブロックの範囲を表します**。{ } や end などの記号・予約語は使いません。
C・Java・PHP（Webサーバ側のプログラム言語）	ブロックの範囲を { } で表します。
Scala	**オブジェクト指向と関数型の両方で書けます**。JVM（Java Virtual Machine、Java仮想マシン）の上で動き、静的型付けです。
R	統計解析とグラフ作成のための、オープンソースの言語と実行環境です。
Kotlin	JVMの上で動きます。Androidアプリの開発言語として使われます。
Go	Googleが開発した、コンパイルして実行する言語です。並行処理が得意です。

Scalaの間違いの選択肢

- 「クラスの多重継承ができる」：ScalaはJavaと同じく、クラスの多重継承はできません。
- 「実行前にネイティブコード（CPU（Central Processing Unit、中央処理装置）が直接実行する機械語）を作る必要がある」：Scalaは通常、JVMのバイトコードにコンパイルして実行します。
- 「動的型付け言語」：Scalaは静的型付けです。動的型付けとは、変数の型を実行時に決める方式のことです（PythonやJavaScript）。

② データの書き方の形式

- **JSON**（JavaScript Object Notation）：JavaScriptのオブジェクトの書き方をもとにしたデータ形式です。「名前と値の組の集まり」と「値の順序付きリスト（配列）」の2つの構造でデータを表します。例：{"name": "田中", "age": 35}
- **XML**（eXtensible Markup Language）：値 のように、タグでデータを囲む形式です。

- **DOM**（Document Object Model）：HTML（HyperText Markup Language、Webページの記述言語）やXMLの文書を、プログラムから木構造として操作するための仕組みです。
- **SOAP**（Simple Object Access Protocol）：XMLでメッセージをやり取りする、Webサービス用の通信規約です。

③ データ項目の命名規約

- 命名規約の内容は次の2つでした。
  - 項目名の末尾に「名」「コード」「金額」などの区分語を付け、区分語ごとにデータ型を決める。
  - 項目名と意味を辞書に登録し、同じ意味の項目を別の名前で作らない。
- この規約で防げること：データ型が混在すること、同じ意味の項目が別名で作られること、項目名から中身が判別できないこと。
- **この規約だけでは防げないこと（正解）**：同じ「受注金額」でも、**取り得る値の範囲が表ごとに違う**ことです。名前と型は決めていても、値の範囲は決めていないからです。

④ メモリのブロック転送

- アドレス1001〜1004のデータを、1003〜1006に移す問題です。転送元と転送先が重なっています。
- 前から順にコピーすると、1001の値を1003に書いた時点で、まだ転送していない元の1003の値を上書きして壊してしまいます。
- そのため、**後ろから順に（1004→1006、1003→1005 …）コピーする**よう方向を指定する必要があります。
- 考え方：転送先が転送元より後ろにずれて重なっているときは、後ろから転送します。

探索（線形・2分・ハッシュ）

① 3つの探索の比較

探索	やり方	前提	速さ
線形探索	先頭から1つずつ順番に比べる	なし	遅い。平均でおよそ n/2 回比べる
2分探索	真ん中と比べ、探す範囲を半分ずつに絞る	**データが整列済み**であること	速い。約 $\log_2 n$ 回
ハッシュ探索	キーから計算で格納場所を直接求める	ハッシュ関数	**データ数に関係なく一定時間**

② ハッシュ探索の時間のグラフ

- 衝突がなければ、データが何件あっても1回の計算で場所が分かります。
- そのため、横軸を「データの個数」、縦軸を「1件の探索時間」にしたグラフは、**横ばいの水平な線**になります（正解）。
- 見分け方：右上がりの線は線形探索（件数に比例）、ゆるやかに増える曲線は2分探索です。

③ ハッシュの衝突の条件

- **ハッシュ関数** $h(x) = x \bmod n$：キー x を n で割った**余り**を、格納場所の番号にします。
- **衝突（シノニム）**：別々のキー a と b が、同じ格納場所になることです。
- 衝突するのは「a を n で割った余り」と「b を n で割った余り」が等しいときです。これは「**a − b が n の倍数**」と同じ意味です（正解）。
- 例：n＝10 のとき、a＝23、b＝13 は、どちらも余りが3なので衝突します。差の 23−13＝10 は、10の倍数です。
- ひっかけ：「a＋b が n の倍数」は、23＋13＝36 で10の倍数ではありません。和ではなく**差**です。

リスト・スタック・キュー

- 配列＝添字で直接アクセスが速いが、挿入・削除で要素の移動が必要。リスト（ポインタ）＝挿入・削除はポインタの付け替えだけで速いが、i番目へのアクセスは先頭からたどる。
- 双方向リスト＝next（次）と prev（前）の両方のポインタを持つ。削除時は前の next と後ろの prev を両方付け替える。
- スタック＝LIFO（Last In First Out、後入れ先出し）。キュー＝FIFO（First In First Out、先入れ先出し）。

2分探索木・AVL木・ヒープ

- 2分探索木＝左の子＜親＜右の子。中間順（左→親→右）でたどると昇順になる。

- 子を2つ持つ節を削除したら、左部分木の最大値（または右部分木の最小値）をその位置に移す。
- AVL（考案者 Adelson-Velsky と Landis の頭文字。平衡2分探索木）木＝左右の部分木の高さの差が1以下になるよう回転で保つ平衡木。探索は O(log n)。
- ヒープ＝親が子より常に大きい（小さい）完全2分木。最大値（最小値）がすぐ取り出せる。

整列（ソート）

- バブルソート＝隣どうしを比べて交換を繰り返す。$O(n^2)$。
- 選択ソート＝未整列部分の最小値を選んで先頭と交換。挿入ソート＝整列済みの部分の正しい位置に挿入（ほぼ整列済みに強い）。
- クイックソート＝基準値（ピボット）より小さい群と大きい群に分けて再帰。平均 O(n log n)、最悪 $O(n^2)$。
- マージソート＝半分に分けて整列し、併合する。常に O(n log n)。ヒープソート＝ヒープを作り最大値を順に取り出す O(n log n)。

コンパイラの処理順

- 字句解析（文字列をトークンに分ける）→構文解析（文法に合うか、構文木を作る）→意味解析（型チェックなど）→最適化→コード生成。

グラフィックス用メモリ・画素

- 必要メモリ＝横画素×縦画素×1画素のビット数÷8［バイト］。動画のデータ量＝1フレームのバイト数×フレーム数/秒×秒数。
- レイトレーシング＝視点から光線を逆にたどり、反射・屈折・影を計算して写実的に描く。

流れ図・アルゴリズムの処理追跡

- 流れ図の問題は、小さな値（M=1, 2, 3）を実際に入れて表を作り、変数の変化を追う。ループの終了条件と、最後に1回多く回らないかを確認する。

03 コンピュータ構成要素

パイプライン・ハザード・VLIW

① パイプライン（pipeline）

- 1つの命令の処理を、いくつかの段階（ステージ）に分けます。そして、前の命令が終わる前に次の命令を始め、少しずつずらして重ねて実行します。
- 例：洗濯を「洗う→干す→たたむ」に分け、1回目を干している間に2回目を洗い始めるのと同じ考え方です。

RISCの5段パイプラインの順序

1. 命令フェッチ：命令をメモリから取り出す
2. 命令デコードとレジスタ読出し：命令の意味を解読する
3. 実行とアドレス生成：計算する
4. メモリアクセス：データを読み書きする
5. 書込み：結果をレジスタに書く

- 覚え方は「取る→読む→やる→メモリ→書く」です。
- RISC（Reduced Instruction Set Computer）：命令の種類を少なく単純にして、1命令を短い時間で実行できるようにしたCPU（Central Processing Unit、中央処理装置）です。パイプラインと相性が良い方式です。
- CISC（Complex Instruction Set Computer）：複雑な命令をたくさん持つCPUです。

② 高速化の技法：問題文の言葉で選ぶ

問題文のキーワード	答え
同時に実行できる複数の動作を、**コンパイルの段階で**まとめて1つの複合命令にする	**VLIW**（Very Long Instruction Word、超長命令語）
分岐の結果が**決まる前に、分岐先を予測して実行**する	**投機実行**
命令を**プログラムの順番どおりではなく**、実行できるものから実行する	アウトオブオーダー実行
分岐命令の直後の命令を、**分岐に関係なく必ず実行**する	遅延分岐
同じレジスタ名を、実行時に**別の物理レジスタに割り当て**て、見かけ上の依存をなくす	レジスタリネーミング
ハードウェアが**実行時に**、並列に実行できる命令を見つけて同時に実行する	スーパースカラ

- VLIWとスーパースカラの違い：並列にできる命令を見つけるのが、VLIWは**コンパイラ（実行前）**、スーパースカラは**ハードウェア（実行時）**です。
- MIMD（Multiple Instruction Multiple Data）：複数のプロセッサが、それぞれ別の命令で別のデータを処理する並列処理の方式です。

③ ハザード（hazard）：パイプラインが止まる原因

- 構造ハザード：同じ装置を、2つの命令が同時に使おうとすることです。
- データハザード：前の命令の結果が出るまで、次の命令が待たされることです。
- 制御ハザード：分岐命令の結果が出るまで、次にどの命令を実行するか決まらないことです。対策が、上の表の投機実行や分岐予測です。

マルチプロセッサとアムダールの法則

計算以外で出るのは、**考え方の問題**です。

① 用語

- **マルチプロセッサ**：1つのコンピュータにCPU（Central Processing Unit、中央処理装置）を複数載せ、1つの処理を分担して並列に実行する方式です。
- **速度向上比**：CPU1台のときと比べて、何倍速くなったかを表す値です。
- **アムダールの法則**（Amdahl's law）：処理の中に**並列化できない部分**（順番に1つずつしか実行できない部分）が残っていると、CPUを何台増やしても、速度向上比には**上限がある**という法則です。

② 例で理解する

- 処理の10%が並列化できないとします。
- CPUを無限に増やすと、並列化できる90%の部分の時間は0に近づきます。それでも、残りの10%の時間は減りません。
- そのため、全体は最大でも「1÷0.1＝10倍」までしか速くなりません。

③ の選択肢の見分け方

選択肢の内容	正誤
並列化**できない**処理がある場合、プロセッサ数を増やしても、速度向上比は**ある水準に近づく（頭打ち）**	○**（正解）**
並列化できない処理がある場合、速度向上比は**プロセッサ数に比例して**増える	×（比例して増えるのは、全部が並列化できるときだけ）
全て並列化できる場合、速度向上比はある水準に近づく	×（全部並列化できるなら、台数に比例して増える）
並列化できる割合が増えると、速度向上比は**反比例して減る**	×（割合が増えるほど、速くなる）

覚え方：「並列化できない部分がある → 頭打ち」。この組合せだけが正解になります。

④ 計算問題に出てくる用語（解かなくても、意味は分かるように）

- **オーバーヘッド**：並列化するための余分な手間です。CPUどうしの連絡や、処理を分ける作業などがあたります。CPUを増やすほど、この手間が増えます。
- **HPC**（High Performance Computing、高性能計算）：スーパーコンピュータのように、大量の計算を高速に行うことです。
- **FLOPS**（FLoating-point Operations Per Second）：1秒間に実行できる、小数を含む計算（浮動小数点演算）の回数です。G（ギガ）は10億、T（テラ）は1兆を表します。

キャッシュメモリ

① キャッシュメモリとは

- CPU（Central Processing Unit、中央処理装置）と主記憶（メインメモリ）の間に置く、小さくて高速なメモリです。
- よく使うデータを手元に置いておき、遅い主記憶まで取りに行く回数を減らします。
- 例：机の上（キャッシュ）と本棚（主記憶）の関係です。よく使う本は机に置いておきます。

② 書込み方式

方式	書き込む先	特徴
ライトスルー（write-through）	キャッシュと主記憶の**両方に同時に**書く	主記憶の内容が**常にキャッシュと一致**する。主記憶への書込みが毎回あるので遅い
ライトバック（write-back）	**キャッシュだけ**に書く。そのデータが**キャッシュから追い出されるときに**、主記憶へ書き戻す	速い。ただし、追い出すときに書き戻しが必要。それまでは主記憶と内容が一致しない

- 覚え方：スルー（through）は「主記憶まで通り抜けて書く」、バック（back）は「後で書き戻す」。
- ひっかけ：「キャッシュだけに書く」「追い出すときに主記憶を変更する」「バスの使用率が低い」は、すべてライトバックの説明です。ライトスルーの説明として出てきたら誤りです。

③ 割当て方式：主記憶のどのブロックを、キャッシュのどこに置けるか

- **ダイレクトマップ**：主記憶の各ブロックが入れる場所は、キャッシュ内の**1か所だけに決まっています**。
- **フルアソシエイティブ**：キャッシュの**どのブロックにも**、主記憶の**どの部分でも**入れられます。
- **セットアソシエイティブ**：上の2つの中間です。主記憶のブロックごとに入れるグループ（セット）が決まり、そのグループの中ならどこにでも入れられます。

④ 置換え方式：キャッシュが満杯のとき、どれを追い出すか

方式	追い出すもの	の表での答え
LRU（Least Recently Used）	最後に使われてから**最も長い時間がたった**もの	**C2**（最終参照 0:05 が最も古い）← **正解**
FIFO（First In First Out、先入れ先出し）	**最も早く読み込んだ**もの	C0（ロード時刻 0:00）
LFU（Least Frequently Used）	**参照回数が最も少ない**もの	C1（参照回数 1）
LIFO（Last In First Out、後入れ先出し）	**最も新しく読み込んだ**もの	C3（ロード時刻 0:05）

- 見分け方：表の列のうち、「最終参照時刻」を見るのがLRU、「ロード時刻」を見るのがFIFOとLIFO、「参照回数」を見るのがLFUです。

ユニファイドメモリ（フレームバッファ）

① 用語

- **フレームバッファ**：画面に表示する1画面分の画像データを置いておくメモリです。画面の各点（画素）の色を記録しています。

- **ユニファイドメモリ**（unified memory、統合メモリ）方式：表示用の専用メモリを持たず、**主記憶（メインメモリ）の一部をフレームバッファとして使う**方式です（正解）。CPU（Central Processing Unit、中央処理装置）とGPU（Graphics Processing Unit、画像処理装置）が、同じ主記憶を共有します。
- 覚え方：unified＝「1つにまとめた」。表示用のメモリと主記憶が1つにまとまっている、と覚えます。

② 特徴

- 長所：専用のメモリが要らないので、**安く、小さく**作れます（ノートPC（Personal Computer）やスマートフォン向き）。
- 短所：画面を表示するために主記憶を読み続けるので、CPUが主記憶を使える時間が減り、**CPUの処理が遅くなることがあります**。

③ 選択肢の見分け方

選択肢	正体
主記憶の一部を表示領域として使う	**ユニファイドメモリ（正解）**
主記憶とは**別に専用の**フレームバッファを持つ	反対の方式（専用のビデオメモリを持つ方式）
シリアル接続した表示装置に、描画コマンドで表示する	小型の表示装置などで使う、別の方式
表示リフレッシュ（画面の書き直し）が不要	電子ペーパーの特徴。ユニファイドメモリとは関係ない

フラッシュメモリ・ウェアレベリング

① フラッシュメモリの特徴

- **フラッシュメモリ**：電源を切ってもデータが消えない（**不揮発性**）メモリです。USB（Universal Serial Bus）メモリ、SSD（Solid State Drive）、スマートフォンの記憶装置に使われています。
- 特徴
  1. 書き換えるときは、**前のデータをまず消去してから**書き込みます（正解）。消去は、ブロックというまとまった単位で行います。
  2. 書換えの回数には**上限があります**（何度も書くと劣化します）。
  3. 基板に付けたまま、電気で書き換えられます。

誤りの選択肢	正しくは
書込み回数は無制限	上限がある
基板から外して、専用のROM（Read Only Memory、読出し専用メモリ）ライターで書く	外す必要はない（外すのは昔のEPROM（Erasable Programmable ROM、紫外線で消去できるROM）という別のメモリ）
定期的にリフレッシュしないとデータが消える	それはDRAM（Dynamic Random Access Memory）の特徴

② ウェアレベリング（wear leveling）

- 書換えが特定のブロックに集中すると、そのブロックだけ早く壊れます。そこで、**各ブロックの書換え回数がなるべく均等になるように書き込む場所を選び、寿命を延ばす**技術です。
- 覚え方：wear（すり減り）＋leveling（平らにならす）＝すり減りを均等にする。

似ている技術との違い

用語	意味
ウェアレベリング	書換え回数を**均等にする**（正解）
不良ブロックの置換え	壊れたブロックを見つけ、予備のブロックに置き換える
多値記録（MLC：Multi Level Cell）	1つのセル（記憶の最小単位）にためる電子の量で、複数ビットを記録する
ジャーナリング	ファイルシステムの変更履歴を先に記録し、障害後に回復できるようにする
デフラグ	ばらばらに置かれたファイルの断片を、連続した場所に並べ直す
ライトアンプリフィケーション	実際に書いた量が、書きたかった量より多くなる現象（消去単位が大きいために起きる）

再入可能プログラム・スレッドセーフ

① プログラムの性質4つ

性質	意味	見分けるキーワード
再入可能（リエントラント、reentrant）	**複数のタスクから同時に呼ばれて並行して実行されても、それぞれに正しい結果を返す**	「複数のタスク」「並行して」
再帰的（リカーシブ、recursive）	手続きの中から**自分自身を呼び出せる**	「自分自身を呼び出す」
再配置可能（リロケータブル、relocatable）	主記憶の**どのアドレスに置いても**実行できる	「どのアドレスから配置しても」
再使用可能（リユーザブル、reusable）	一度実行した後、読み込み直さずに**もう一度使える**。ただし同時には使えない	「再ロードせずに」「逐次」

- タスク：OS（Operating System）から見た、1つの仕事の単位です。
- 再入可能にするしくみ：命令の部分（全タスクで共有）と、データの部分（呼び出したタスクごとに別々に持つ）を分けます。データが混ざらないので、同時に呼ばれても結果が壊れません。
- もう1つのひっかけ：「必要な部分を補助記憶装置から読み込みながら動作する」は、**オーバーレイ**という別の方式の説明です。

② スレッドセーフ

- **スレッド**：1つのプログラムの中で、並行して動く処理の流れです。
- **スレッドセーフ**：**複数のスレッドで並列に実行しても、問題が起きない**ことです（正解）。
- 実現のしかた：共有する変数を更新している間は、**排他制御**（同時に1つのスレッドしか触れないようにする仕組み）で守ります。手段はミューテックスやセマフォです（3/71で扱いました）。

誤りの選択肢	誤りの理由
複数のスレッドから呼ばれないようにする	呼ばれても安全なのがスレッドセーフ。呼ばせないことではない
単一のスレッドで動かす	並列に動かしても安全なのがスレッドセーフ
競合が起きたら安全に止める	止めるのではなく、問題を起こさないことがスレッドセーフ

- 覚え方：再入可能は「タスク」、スレッドセーフは「スレッド」。どちらも「**同時に動かしても正しい**」という同じ考え方です。

CPU性能（クロック・CPI・MIPS）

- 1命令の平均実行時間＝CPI（Cycles Per Instruction、1命令あたりのクロック数）÷クロック周波数。
- MIPS（Million Instructions Per Second）＝クロック周波数（MHz）÷CPI。例：1.25GHz で 1秒に10億命令→CPI＝1.25。

スマートフォン向けプロセッサ（big.LITTLE 等）

- 高性能コアと省電力コアを組み合わせ、負荷に応じて使い分けて性能と消費電力を両立する（ヘテロジニアス・マルチコア）。

量子コンピュータ

- 量子ビット（qubit）＝0と1の重ね合わせ状態を持てる。量子ゲート方式＝量子ゲートを組み合わせて汎用計算を行う（量子アニーリング方式は組合せ最適化に特化）。
- 量子超越性（Quantum Supremacy）＝量子コンピュータが、現実的な時間では古典コンピュータで解けない問題を解けることを示すこと。

DRAM・SRAM（リフレッシュ）

- DRAM（Dynamic RAM（Random Access Memory、読み書きできるメモリ））＝コンデンサに電荷をためて記憶する。電荷が漏れるので、定期的なリフレッシュ（再書込み）が必要。安くて大容量なので主記憶に使う。

- SRAM（Static RAM）＝フリップフロップで記憶する。リフレッシュ不要で高速だが高い。キャッシュメモリに使う。
- どちらも揮発性（電源を切ると消える）。フラッシュメモリは不揮発性。

入出力インタフェース（FC・PLC・I2C）

- FC（Fibre Channel）＝サーバとストレージをつなぐ高速インタフェース。SAN（Storage Area Network）で使う。
- PLC（Programmable Logic Controller）＝工場の機械をラダー図などのプログラムで制御する装置。（通信の PLC＝電力線通信 Power Line Communication と区別）
- I2C（Inter-Integrated Circuit）＝2本の信号線（クロックとデータ）で複数の機器をつなぐシリアルバス。機器はアドレスで区別し、同時に1つしか通信できない。

入出力装置・表示装置

- 電気泳動型電子ペーパー＝帯電した粒子を電圧で動かして表示する。表示の保持に電力が要らない（書換え時だけ電力を使う）。
- 有機EL（Electro Luminescence）＝素子自体が発光する。バックライト不要で薄く、黒がきれい。液晶はバックライトが必要。
- データセンターの直流給電＝交流と直流の変換の回数が減り、変換で失う電力（損失）が減る。
- CPU（Central Processing Unit、中央処理装置） とディスクの使用率：ジョブの CPU 時間とディスク時間を時間軸に並べ（同時に使えるか条件を確認）、全体の経過時間に対する使用時間の割合を出す。
- VR（Virtual Reality）のレンダリング＝3次元データから画像を生成すること。

アクチュエーター

- アクチュエーター＝電気などのエネルギーを、回転・直線運動などの機械的な動きに変える装置（モーター、ソレノイドなど）。センサーは物理量を「測る」側、アクチュエーターは「動かす」側。

04 システム構成要素

IaC・プロビジョニング

① IaC（Infrastructure as Code、コードとしてのインフラ）

- **定義**：サーバやネットワークの**構成や設定をプログラム（コード）として書いておき、専用のソフトウェアがその内容どおりに自動でシステムを組み立てる**ことです。
  - インフラ（インフラストラクチャ）とは、サーバ・ネットワーク・ストレージなど、システムの土台のことです。
- 例：「Webサーバを3台、それぞれメモリ8GB、このソフトを入れる」と設定ファイルに書いておくと、ツールが自動で構築します。
- **長所**：同じ環境を何度でも正確に作れます。手作業のミスがなくなります。設定の変更履歴をバージョン管理できます。
- 正解の決まり文句
  - 「システムの**構成や状態をコードに定義**すること」
  - 「構成・設定を**プログラムとして記述**し、専用のソフトウェアが**自動的に**適用する」

ひっかけの選択肢

選択肢	誤りの理由
インシデント（障害）への対応手順をコードに書く	対象が障害対応で、構成ではない
アプリの生成手順や試験の手順をコードに書く	CI（Continuous Integration、継続的インテグレーション）の説明
ツールの連携手順をコードに書く	対象が構成ではない
何も入っていない初期状態のサーバ	ベアメタルサーバの説明
資源の割当てや設定を**手動で**行う	IaCは**自動**

② プロビジョニング（provisioning）

- **定義**：利用者の需要を予想して、サーバやネットワークなどの資源を**あらかじめ用意し、すぐ使える状態に整えておく**ことです。
  - 覚え方：provision＝「備え・準備」。
- **サーバプロビジョニングツール**：**システムの構成をあらかじめ書いておき、サーバを自動で構成する**ツールです（正解）。
- クラウドのプロビジョニング：利用者の需要を予想し、設備や資源を計画的に用意して強化し、利用者の求めに応じてサービスを提供できるよう備えることです（正解）。

ひっかけ

- サービスが動いているかを遠隔で監視する、性能を監視・管理する：どちらも監視ツールの説明です。
- インストールしたソフトを一元管理する：構成管理（資産管理）ツールの説明です。

ひっかけ

- 企画から保守まで一貫して提供する：SI（System Integration、システムインテグレーション）の説明です。
- 複数の企業が共同でデータセンターを運用する：コミュニティクラウドの説明です。
- 自社で機器を用意してパッケージを導入する：オンプレミスの説明です。

信頼性設計

① 信頼性設計の5つの考え方

用語	英語の意味	定義	例
フォールトアボイダンス	fault（故障）＋avoidance（回避）	部品**一つ一つの品質を高めて、故障そのものが起きないようにする**	高品質な部品を使う
フォールトトレランス	fault＋tolerance（耐える）	重要な部品を**多重化して、故障が起きても動き続ける**	サーバを2台用意し、1台が壊れても動き続ける
フェールセーフ	fail（故障）＋safe（安全）	故障したら、**安全な側に止める**	信号機が故障したら全部赤にする
フェールソフト	fail＋soft（柔らかく）	故障したら、**機能を落としてでも動き続ける（縮退運転）**	エンジンが1つ止まっても、残りで飛び続ける
フールプルーフ	fool（うっかり者）＋proof（耐える）	**人が誤った操作をしても**危険にならないようにする	ドアを閉めないと電子レンジが動かない

正解の文

- 「フォールトアボイダンスとは、構成要素の品質を高めて故障が発生しないようにする」
- 「フォールトトレランスは、重要部品を多重化して故障に備える」

ひっかけのパターン：説明と名前の入れ替え

誤りの選択肢	実際に当てはまる用語
「フェールセーフとは、利用者の誤操作でミスが起きないようにする」	フールプルーフ
「フェールソフトとは、機能を縮退**させることなく**動き続ける」	縮退**して**動くのがフェールソフト
「フォールトトレランスとは、安全状態にシステムを固定する」	フェールセーフ
「フェールセーフは、部品の信頼性を高めて故障を起こさない」	フォールトアボイダンス
「フォールトアボイダンスは、故障しても運用を続ける」	フォールトトレランス

② 稼働率の意味

- **MTBF**（Mean Time Between Failures、平均故障間隔）：**直ってから次に故障するまで**平均時間、つまり正常に動いている時間の平均です。
- **MTTR**（Mean Time To Repair、平均修理時間）：**故障してから直るまで**平均時間です。
- **稼働率 ＝ MTBF ÷（MTBF ＋ MTTR）**：システムが**動いている時間の割合**です。

誤りの選択肢	誤りの理由
「MTBF − MTTR は、正常だった時間」	正常だった時間はMTBFそのもの
「MTBFは、運用を始めてから**初めて**故障するまでの時間」	故障と故障の間の**平均**時間
「MTTRは、回復してから次の故障までの時間」	それはMTBF。MTTRは修理にかかる時間

MTBF・MTTR

① 略語と意味

- **MTBF**（Mean Time Between Failures、平均故障間隔）：直ってから次に故障するまでの平均時間です。**長いほど、故障しにくい**ことを表します。
- **MTTR**（Mean Time To Repair、平均修理時間）：故障してから直るまでの平均時間です。**短いほど、早く直る**ことを表します。

② どちらに効く施策か

- 問題は「MTBFを長くするよりも、MTTRを短くするのに役立つものはどれか」です。
- 見分け方
  - 故障を**起こさない・起きても止まらない**ようにする施策は、MTBFを長くします。
  - 故障した後に**早く見つけて早く直す**施策は、MTTRを短くします。

選択肢	どちらに効くか	理由
エラーログ取得機能	**MTTR（正解）**	故障の原因と場所がすぐ分かり、修理が早くなる
記憶装置のビット誤り訂正機能	MTBF	誤りを自動で直すので、故障として表に出ない
命令再試行機能	MTBF	一時的な誤りを、もう一度実行して回避する
予防保全	MTBF	壊れる前に部品を計画的に交換する

- MTTRを短くする施策の仲間：エラーログ、遠隔保守（離れた場所から直す）、自己診断機能、保守部品を手元に置いておくこと、障害の自動通知。

③ 稼働率の決まり

- 稼働率 ＝ MTBF ÷（MTBF ＋ MTTR）
- **MTBFとMTTRが同じ倍率で変わると、稼働率は変わりません。**分母と分子が同じ倍率で変わるからです。
  - 例：100÷(100＋10) と 150÷(150＋15) は、どちらも約0.909です。
- **2つのシステムの稼働率が同じなら、「MTTR÷MTBF」の値も同じです。**ただし、MTBFとMTTRがそれぞれ同じ値だとは限りません。
  - 例：MTBF 100・MTTR 10 と、MTBF 200・MTTR 20 は、稼働率は同じですが、それぞれの値は違います。
- MTTRだけを短くすると、稼働率は上がります。

スケールアップ・スケールアウト・キャパシティプランニング

① 4つの「スケール」

用語	何を変えるか	増やす／減らす
スケールアップ	**1台の性能**（CPU（Central Processing Unit、中央処理装置）やメモリを強化）	増やす
スケールダウン	1台の性能	減らす
スケールアウト	**サーバの台数**	増やす
スケールイン	**サーバの台数**	**減らす**

- 覚え方：**アップ・ダウンは「1台の性能」**上げ下げ（縦方向）です。**アウト・インは「台数」**増減（横方向）です。
- ひっかけ
  - 「CPUの能力を減らす」はスケールダウンです。

- 「台数を増やす」はスケールアウトです。
- 「メモリを増やす」はスケールアップです。

② スケールアウトが向いているシステム

- 正解：**参照系（データを読むだけ）のトランザクションが多く、複数のサーバで分散処理しているシステム**です。
- 理由：読むだけの処理は、サーバどうしでデータを合わせる必要がありません。そのため、台数を増やせばそのまま処理能力が上がります。
- 向いていないシステム（どれもスケールアップ向き）
  - 並列に処理しにくい、大きな一括処理が中心のシステム（分けられないので、台数を増やしても速くならない）
  - データの追加（更新）が多く、整合性をとる手間を小さくしたいシステム（台数を増やすと、整合性をとる手間が増える）
  - 同じマスターデータベースを複数置き、更新のたびに整合性をとる必要があるシステム（同じ理由）

③ キャパシティプランニング

- **定義**：システムの処理能力（キャパシティ）について、**今の使用状況を調べて長く監視し、将来の需要を予測して、必要な資源を計画的に用意する**ことです。
- 正解：「現在の応答時間を調査して長期的に監視し、**将来を含めて**応答時間を維持すること」。
- 見分け方：キーワードは「**将来**」「**長期的**」です。

誤りの選択肢	正体
最も影響の大きいボトルネック（処理の滞る箇所）だけに注目して取り除く	ボトルネック分析（今の問題を解決するだけ）
ソフトとハードをチューニングして、**今の**能力を最大限に引き出す	性能チューニング
特定の資源の使い方の効率を上げる	資源の使い方の改善

- これらは「今の性能を良くする」話で、「将来に備える」話ではないので誤りです。

RAID

- RAID0＝ストライピング（分散して速いが冗長なし）。RAID1＝ミラーリング（容量は半分）。
- RAID5＝データとパリティを全ディスクに分散。1台故障まで復旧できる。実効容量＝(台数−1)×1台の容量。RAID6＝パリティ2つで2台故障まで。容量は(台数−2)台分。
- ホットスペア（予備ディスク）は容量に含めない。

仮想化（コンテナ型・ハイパーバイザ型）

- コンテナ型＝ホストOS（Operating System、基本ソフト）のカーネルを共有し、アプリと必要なライブラリだけをコンテナに分けて動かす。起動が速く軽い。ゲストOSはない。
- ハイパーバイザ型＝ハードウェアの上でハイパーバイザが動き、その上に複数のゲストOSを動かす。
- ホスト型＝ホストOSの上で仮想化ソフトが動き、その上にゲストOS。

クライアントサーバ・ストアドプロシージャ・プロキシ

- ストアドプロシージャ＝一連の SQL（Structured Query Language、データベースを操作する言語） を DB（Database、データベース）サーバに登録しておき、クライアントは名前を呼ぶだけで実行する。通信回数が減ってネットワークの負荷が下がる。
- 3層クライアントサーバ＝プレゼンテーション層（画面）・ファンクション層（処理）・データ層（DB）に分ける。

オブジェクトストレージ

- データを「オブジェクト（データ本体＋メタデータ＋一意のID（Identification、識別子））」として階層のないフラットな空間に格納し、HTTP（REST（Representational State Transfer、HTTPで資源を操作する設計方式） API（Application Programming Interface、ソフトどうしの呼出し口））でアクセスする。大量データの保存と拡張に向く。
- ブロックストレージ＝ディスクをブロック単位で扱う（DB（Database、データベース）向け）。ファイルストレージ＝フォルダ（階層）で管理する。

稼働率（直列・並列）

- 稼働率＝MTBF（Mean Time Between Failures、平均故障間隔） / (MTBF＋MTTR（Mean Time To Repair、平均修理時間))。
- 直列：全部動いて動く＝A1×A2。並列：どれか1つ動けば動く＝1−(1−A1)(1−A2)。
- 3台中2台以上で動く（2 out of 3）＝$3A^2-2A^3$。

API仕様（OpenAPI）

- OpenAPI Specification＝REST（Representational State Transfer、HTTPで資源を操作する設計方式） API（Application Programming Interface、ソフトどうしの呼出し口） の仕様を YAML（YAML Ain't Markup Language、設定ファイルの書式） や JSON（JavaScript Object Notation、データ形式） で記述する標準。仕様書からドキュメントやコードを自動生成できる。

05 ソフトウェア

排他制御・デッドロック・セマフォ

- 正解は**セマフォ**です。選択肢はキュー、マルチスレッド、メモリプールでした。
- 排他制御：1つの資源を、同時に1つのタスクだけが使えるようにすることです。
- I2C（Inter-Integrated Circuit、2本の線のシリアルバス）バス：信号線が2本だけの、機器どうしをつなぐ通信路です。同時に1つの機器としか通信できないので、排他制御が必要になります。
- セマフォ：使える資源の数を表す変数です。
  - P操作（確保）：値を1減らします。値が0のときは、空くまで待ちます。
  - V操作（解放）：値を1増やします。
  - 例：バスが1本なら初期値は1です。タスクAがP操作をすると0になり、タスクBはAがV操作をするまで待ちます。
- ミューテックス（mutex）：資源が1つの場合の排他制御の仕組みです。ロックした本人しか解除できない点が、セマフォと違います。
- ほかの選択肢との違い
  - キュー（メールボックス）：タスクどうしでデータを受け渡す仕組みで、排他制御ではありません。
  - メモリプール：決まった大きさのメモリ領域を貸し出す仕組みです。
  - マルチスレッド：処理を並行して実行することで、排他制御の手段ではありません。
- 正解は**資源を確保する順序を、両方のタスクで同じにする**ことです。
- デッドロック：2つ以上のタスクが、互いに相手の持っている資源が空くのを待ち、どちらも進めなくなる状態です。
  - 例：タスク1は資源Xを持ったまま資源Yを待つ。タスク2は資源Yを持ったまま資源Xを待つ。こうして永久に止まります。
  - 両方が「X→Yの順」に確保するなら、Xを取れなかった方はYにも手を出しません。そのため、互いに待ち合う状態にはなりません。
- 間違えやすい選択肢：「順序を逆にする」は、まさにデッドロックが起きる条件です。「優先度を高くする」「優先度を同じにする」では、互いに待ち合う状態は解消しません。

パターン3：資源の確保順の表から、デッドロックの可能性を探す

- 3つの資源を4つのプロセスが確保する順番の表が出ます。プロセスAと組み合わせたときに、デッドロックを起こす可能性があるプロセスを選ぶ問題です。正解はC、Dでした。
- 判定のしかた：Aと比べて、**2つの資源の確保順が逆になっている組合せがあれば、デッドロックの可能性があります。**たとえばAが「X→Y」、相手が「Y→X」の場合です。

メモリ管理（ベストフィット・断片化）

① 空き領域の割り当て方3つ

プログラムからメモリを要求されたとき、空いている領域のうち、どれを渡すかの決め方です。

方式	どの空き領域を渡すか	結果としての傾向
ファーストフィット（first-fit、最初適合）	先頭から探して、**最初に見つかった**足りる領域	アドレスの小さい側（下位）がよく使われるので、アドレスの大きい側（上位）に大きな空きが残る
ベストフィット（best-fit、最適適合）	足りる領域のうち、**最も小さい**領域	余りがわずかずつ出るので、最終的に**小さな空き領域がたくさん残る**
ワーストフィット（worst-fit）	**最も大きい**領域	大きな空き領域がなくなっていく

- ひっかけ
  - 「下位のアドレスをよく使い、上位に大きな空きが残る」は、ファーストフィットの特徴です。
  - 「ハッシュ関数で検索する」「アドレス順に管理し、隣の空きと簡単にまとめられる」は、どちらもベストフィットとは関係ありません。

② ベストフィットに向いたデータ構造

- ベストフィットでは、「要求量以上で最も小さい空き領域」を素早く見つける必要があります。
- そのためには、**空き領域の大きさをキーにした2分探索木**が最も速く探せます（正解）。2分探索木は、値の大小で左右に分けて並べた木の形のデータ構造です。
- ひっかけ
  - 「アドレスをキーにした2分探索木」：大きさで探せないので誤りです。
  - 「大きさの順の片方向リスト」：先頭から順にたどるので遅く、誤りです。
  - 「ビットマップ」：大きさで探せないので誤りです。

③ フラグメンテーション（断片化）

- **定義**：割当てと解放を繰り返すうちに、空き領域が細切れになることです。その結果、**空きの合計は十分あるのに、必要な大きさのまとまった領域が確保できなくなる**ことがあります（正解）。
- 対策は**コンパクション**（ガベージコレクション）です。使っている領域を寄せて詰め、空きを1か所にまとめます。
- ひっかけ
  - 「可変長ブロックなら断片化は起きない」：可変長ブロックこそ断片化が起きるので誤りです。
  - 「固定長ブロックは断片化が起きやすい」：固定長は大きさがそろっているので起きにくく、誤りです。
  - 「返却のたびに毎回ガーベジコレクションを行う必要がある」：毎回は不要で、処理が重くなるだけなので誤りです。

タスクの状態とスケジューリング

① 用語

- **タスク**：OS（Operating System、基本ソフト）から見た、1つの仕事の単位です。
- **スケジューリング**：どのタスクにCPU（Central Processing Unit、中央処理装置）を使わせるかを決めることです。
- **プリエンプティブ**（preemptive）：OSが、実行中のタスクを**途中で強制的に中断して**、優先度の高いタスクにCPUを渡せる方式です。英語の preempt は「横取りする」という意味です。
- **ノンプリエンプティブ**：OSは強制的に中断できない方式です。**実行中のタスクが自分から中断を申し出たときだけ**、ほかのタスクに切り替わります。

② タスクの3つの状態

状態	意味
実行状態	今CPUを使っている
実行可能状態	すぐ動けるが、CPUが空くのを待っている
待ち状態	入出力の完了など、何かの出来事を待っている（CPUがあっても動けない）

状態の移り方

きっかけ	移り方
CPUが割り当てられた	実行可能 → 実行
横取り（プリエンプション）された	**実行 → 実行可能**
入出力などを待つ	実行 → 待ち
待っていた出来事が起きた	待ち → **実行可能**（実行に直接は戻らない）

③ 割込み処理

- **割込み**：キー入力やタイマーなど、ハードウェアからの合図で、今の処理を一時中断させる仕組みです。
- **割込み処理ルーチン**は、**どんなに優先度の高いタスクよりも優先して実行されます**（正解）。

誤りの選択肢	誤りの理由
「タスクの切替えを禁止すると、割込み処理ルーチンは呼ばれない」	割込み処理ルーチンはタスクではないので、切替えを禁止しても呼ばれる
「割込み処理ルーチンの処理時間は、応答性に影響しない」	長いと、その間ほかの処理が止まるので影響する
「割り込まれたタスクと同じコンテキスト（実行環境）で動く」	別のコンテキストで動く

④ いつまでも待たされるタスクが出る方式

- 正解：**処理予定時間が短いタスクから順に実行する方式**（SPT：Shortest Processing Time、処理時間順方式）です。短いタスクが次々に到着すると、長いタスクはいつまでも後回しになります。これを**スタベーション**（飢餓状態）といいます。

ほかの選択肢	方式の名前	待たされ続けない理由
待ち時間に応じて優先度を少しずつ上げる	エージング	待つほど優先度が上がる
一定時間ごとに中断して列の最後に回す	ラウンドロビン	必ず順番が回ってくる
到着した順に実行する	FCFS（First Come First Served、到着順）	順番どおりに必ず実行される

仮想記憶・ページング・置換え

- ページング＝プログラムを固定長のページに分けて主記憶に割り当てる。ページフォールト＝必要なページが主記憶にないこと。
- 置換え方式：FIFO（最も早く読み込んだページを追い出す）、LRU（最も長く参照されていないページを追い出す）、LFU（参照回数が最も少ないページ）。
- スラッシング＝ページの入れ替えが多発して、処理がほとんど進まない状態（主記憶が足りないとき）。
- セグメンテーション＝論理的なまとまり（可変長）で分ける。ページング＝固定長。
- ページの内部断片化：プログラムの最後のページに空きができる（平均で半ページ）。

開発ツール（IDE・クロス開発・プロファイラ等）

- IDE（Integrated Development Environment、統合開発環境）＝エディタ・コンパイラ・デバッガなどを1つにまとめた環境。
- クロスコンパイラ＝開発に使うコンピュータとは異なるアーキテクチャのコンピュータで動く機械語を生成する（組込み開発）。
- プロファイラ＝モジュールや関数の実行回数・実行時間を測り、性能のボトルネックを見つける。
- ドキュメンテーションジェネレーター＝ソースコードのコメントなどから仕様書を自動生成する。
- エミュレーター＝別のハードウェアの動作をまねて、そのプログラムを実行する。

SAN・NAS

- SAN（Storage Area Network）＝サーバとストレージを専用のネットワーク（FC（Fibre Channel、ストレージ用の高速インタフェース） など）でつなぎ、ブロック単位でアクセスする。

- NAS（Network Attached Storage）＝LAN（Local Area Network、構内ネットワーク） につなぐファイルサーバ。ファイル単位でアクセスする（NFS（Network File System、ファイル共有の規約）、SMB（Server Message Block、Windowsのファイル共有の規約））。

OSS・Linux・ライセンス

- OSS（Open Source Software）＝ソースコードが公開され、改変・再配布できる。
- Linux カーネル＝OS（Operating System、基本ソフト） の中核。GPL（GNU（GNU's Not Unix、自由ソフトウェアの計画） General Public License）で配布。
- GPL＝改変したものを配布するときはソースコードを公開し、同じ GPL にする（コピーレフト）。BSD（Berkeley Software Distribution、緩いOSSライセンス）・MIT＝著作権表示を残せば、改変物のソースを公開しなくてよい。

Git（分散バージョン管理）

- 分散型バージョン管理＝各開発者のローカル環境にリポジトリ（変更履歴の保管場所）の完全な複製を持つ。
- commit＝ローカルのリポジトリに変更を記録する。push＝ローカルの変更をリモート（共有）リポジトリに反映する。pull＝リモートから取り込む（fetch＋merge）。clone＝リポジトリを丸ごと複製する。

クラスタ構成（ホットスタンバイ等）

- ホットスタンバイ＝待機系を起動したままにしておき、障害時にすぐ切り替える。コールドスタンバイ＝待機系は停止しておき、障害時に起動する（切替えが遅い）。
- アクティブ／アクティブ＝両方で処理を分担。アクティブ／スタンバイ＝片方だけが処理し、もう片方は待機。

06 ハードウェア

電子部品（ダイオード・MOS・インバーター）

① 電子部品の働き

部品	働き
ダイオード	**電流を一方向にだけ流す（整流作用）**。交流を直流に変えるのに使う
コンデンサ	電気をためたり放したりする
コイル	電流の変化を妨げる（磁気のエネルギーをためる）
抵抗器	電流を流れにくくして、量を調整する

- **整流**：向きが交互に変わる交流を、一方向だけの流れ（直流）にすることです。
- **図の問題**：ダイオードを通すと、波形の**片側（プラスかマイナスの一方）だけが残り、反対側は0になります**。これを読み取れば解けます。

② 半導体素子の見分け方

半導体素子とは、電気を通す・通さないを制御できる材料（半導体）で作った部品のことです。

説明	素子
金属と半導体の間に、酸化物の絶縁体をはさんだ構造	**MOSトランジスタ**（Metal-Oxide-Semiconductor：金属・酸化物・半導体の頭文字）（正解）
電子と正孔がくっつくときに**光を出す**	LED（Light Emitting Diode、発光ダイオード）
光が当たると電流が流れる	フォトダイオード
逆向きの電圧を一定以上かけると、**電圧がほぼ一定に保たれる**	ツェナーダイオード（定電圧ダイオード）

- 覚え方：MOSという名前そのものが、構造（金属・酸化物・半導体）を表しています。

③ インバーター

- **インバーター：直流の電力を交流の電力に変える**装置です（正解）。交流の周波数を変えられるので、エアコンや電気自動車のモーターの回転の速さを細かく制御でき、省エネになります。
- パワー半導体：大きな電力を制御・変換するための半導体です。

変換の向き	名前
直流 → 交流	**インバーター（正解）**
交流 → 直流	コンバーター（整流器）
直流 → より高い／低い直流	DC-DCコンバーター（DC：Direct Current、直流）

- 覚え方：invert＝「逆にする」。ふつうは交流を直流に変えるので、その逆（直流 → 交流）がインバーターです。

④ LED回路の図

- 「LEDが点灯する条件」は、スイッチAとBの組合せで、LEDに電流が流れる道があるかどうかで決まります。
- 2つのスイッチが直列（一列）につながっていればAND（両方オンで点灯）、並列につながっていればOR（どちらかがオンで点灯）になります。図を見て、どちらかを判断してください。

FPGA・HDL・SoC

- FPGA（Field Programmable Gate Array）＝製造後に利用者が論理回路の構成を書き換えられる LSI（Large Scale Integration、大規模集積回路）。
- HDL（Hardware Description Language、ハードウェア記述言語）＝Verilog・VHDL（ハードウェア記述言語の一つ） など。デジタル回路を記述し、論理合成で回路にする。
- SoC（System on a Chip）＝CPU（Central Processing Unit、中央処理装置）・メモリ・入出力など、システムに必要な機能を1つのチップにまとめたもの。SiP（System in Package）＝複数のチップを1つのパッケージに収めたもの。
- ASIC（Application Specific Integrated Circuit）＝特定用途向けに設計・製造する LSI（後から変えられない）。

耐タンパ性

- 耐タンパ性（tamper resistance）＝内部の情報を不正に読み出したり改ざんしたりすることへの抵抗力。
- 手法：チップの内部を物理的に解析されないようにする、解析されたら内部の情報を消去する、配線を複雑にする、消費電力や処理時間から秘密が漏れないようにする（サイドチャネル攻撃の対策）。

DMA

- DMA（Direct Memory Access）＝CPU（Central Processing Unit、中央処理装置） を介さずに、入出力装置と主記憶の間でデータを直接転送する。CPU は転送の間ほかの処理ができる。

07 ユーザーインタフェース

ユーザビリティ評価

- 思考発話法＝利用者に操作してもらいながら、考えていることを声に出してもらい、問題点を見つける（利用者が参加する）。
- ヒューリスティック評価＝専門家が経験則（ガイドライン）に照らして問題点を見つける（利用者は参加しない）。
- 認知的ウォークスルー＝専門家が利用者になったつもりで、操作の手順を1つずつたどって評価する。
- 使用性（非機能要件）＝習得のしやすさ・操作のしやすさ。例：4時間のトレーニングで操作できる。

画面の部品（カルーセル・ハンバーガーメニュー等）

- カルーセル＝複数の画像やコンテンツを、横に回転するように切り替えて同じ場所に表示する部品。
- ハンバーガーメニュー＝三本線のアイコン。押すとメニューが開く（画面が小さいときに使う）。
- パンくずリスト＝今いるページの階層をたどる表示。

- レスポンシブWebデザイン＝1つの HTML（HyperText Markup Language、Webページの記述言語） を、画面の幅に応じて CSS（メディアクエリ）でレイアウトを変えて表示する。

コード設計・フォント

- コードの桁数：26種の英字を n桁使うと 26^n 通り。必要な件数を超える最小の n を求める。
- アウトラインフォント＝文字の輪郭を線と曲線の式で持つ。拡大しても滑らか。ビットマップフォント＝点の集まりで、拡大するとギザギザになる。

08 情報メディア

動画・画像・音声の形式

① H.264/MPEG-4 AVC

- **動画のデータを圧縮する方式**（圧縮符号化方式）です。ワンセグ放送、ブルーレイ、インターネットの動画配信で使われています（正解）。
  - MPEG：Moving Picture Experts Group。動画の規格を作る国際的な団体の名前です。
  - AVC：Advanced Video Coding。高度な動画符号化という意味です。
- ほかの選択肢の正体 | 選択肢の説明 | 正体 | |---|---| | ストリーミング配信を制御する通信方式 | RTSP（Real Time Streaming Protocol） | | テレビ会議の双方向ビデオを制御する通信方式 | H.323 や SIP（Session Initiation Protocol） | | テレビの番組表に使う、番組内容のメタデータ（データについての情報）の記述方式 | EPG（Electronic Program Guide、電子番組表）のメタデータ |
- 見分け方：H.26x や MPEG は「**圧縮**」の規格です。「通信の制御」の規格ではありません。

② 画像の形式

形式	正式名	特徴
SVG	Scalable Vector Graphics	**W3C**（World Wide Web Consortium、Webの標準を決める団体）が定めた形式。四角形・円・直線・文字などの**図形を****XML**（eXtensible Markup Language）**で記述する**。拡大しても劣化しない（**正解**）
PNG	Portable Network Graphics	点（画素）の集まりで記録する形式。圧縮しても画質が落ちない（可逆圧縮）
TIFF	Tagged Image File Format	点の集まりで記録する形式。印刷用などに使う
OpenGL	Open Graphics Library	画像の形式ではない。3次元の図形を描画するためのプログラムの仕組み（API：Application Programming Interface）

- 見分け方：「図形を**XMLで記述**」「W3C」とあれば **SVG** です。

③ 標本化定理

- アナログの音声をデジタルにするときは、**元の信号の最高周波数の2倍以上**頻度で値を読み取れば、元の音を再現できます。
- 例：20kHzまでの音なら、1秒間に4万回（40kHz）以上読み取ります。

コンピュータグラフィックス

- レンダリング＝3次元のデータから画像を生成する。テクスチャマッピング＝物体の表面に画像を貼り付けて質感を出す。
- レイトレーシング＝視点から光線をたどって反射・屈折を計算する。Zバッファ法＝画素ごとに奥行きを比べて手前の面だけを描く（隠面消去）。
- アンチエイリアシング＝斜めの線のギザギザを目立たなくする。

09 データベース

同時実行制御（ロック・MVCC・2相コミット）

前提：トランザクション（transaction）

- データベースへの一連の更新を、1つのまとまりとして扱う単位です。
- 例：振込は「Aの口座から1万円引く」と「Bの口座に1万円足す」の2つの更新で、この2つで1つのトランザクションです。

① ロック（lock）：同じデータを同時に書き換えて壊さないための鍵

共有ロック（読取り用）

- 自分が読んでいる間は、ほかのトランザクションも**読めます**。ほかの人も共有ロックを掛けられます。
- ほかのトランザクションは**書けません**（専有ロックは掛けられません）。
- 例：「T2も同じデータに共有ロックを掛けて読める」でした。

専有ロック（排他ロック、書込み用）

- 自分が書いている間は、ほかのトランザクションは**読むことも書くこともできません**。

表：先に掛かっているロック（行）と、後から掛けようとするロック（列）

	後から共有ロック	後から専有ロック
先に共有ロック	○ 掛けられる	× 待つ
先に専有ロック	× 待つ	× 待つ

2相ロッキング

- ロックを掛けていく段階と、外していく段階を分ける方式です。一度外し始めたら、新しいロックは掛けません。
- 目的は、並行して実行しても、1つずつ順番に実行したのと同じ結果になることを保証することです。これを直列化可能性といいます。

② MVCC（Multi-Version Concurrency Control、多版同時実行制御）＝の正解

- 同じデータについて、**更新前の版と更新後の版を両方持っておく**仕組みです。
- 書き込んでいる最中でも、読むトランザクションは更新前の版を読めます。そのため**読取りが書込みを待たずに済み**、同時に実行できる処理が増えます。
- 見分け方：問題文に「異なる版」「版を用意」とあれば MVCC です。
- ほかの選択肢
  - 2相ロッキングプロトコル：ロックで直列化可能性を保証する方式で、版は作りません。
  - WAL（Write-Ahead Logging、先書きログ）：データを書き換える前に、変更内容をログに書いておく障害回復の方式です。
  - 共有ロック：読取り用のロックです。

③ 2相コミット（two-phase commit）：複数のDBにまたがる更新を、全部確定するか全部取り消すか

- 1相目：調整役が、すべてのDB（Database、データベース）に「確定（コミット）できるか」を問い合わせます。
- 2相目：全員が「できる」と答えたときだけ、全員に一斉にコミットを指示します。1つでも「できない」があれば、全員に取消し（ロールバック）を指示します。
- 目的は**原子性**（全部実行するか、全く実行しないか）を、複数のDBの間で守ることです。
- 弱点：1相目の後に調整役が止まると、各DBは確定も取消しもできないまま待ち続けます。これをブロックといいます。
- 「2相ロッキング」（ロックの方式）と「2相コミット」（分散DBの確定の方式）は、名前が似ていますが別物です。

④ ACID特性（トランザクションが守る4つの性質）

- A＝原子性（Atomicity）：全部実行するか、全く実行しないか。
- C＝一貫性（Consistency）：データの整合性が保たれる。
- I＝独立性（Isolation）：同時に動くトランザクションどうしが影響し合わない。
- D＝耐久性（Durability）：確定した結果は、障害が起きても消えない。

- 「可用性」はACID（Atomicity・Consistency・Isolation・Durability、トランザクションの4性質）に入りません。

ACID・DDoS・MTBF・MTTR の正式名

- **ACID**（Atomicity・Consistency・Isolation・Durability、トランザクションの4性質）：トランザクション（データベースへの一連の更新を1つにまとめた単位）が守る、4つの性質の頭文字です。
  - **A**tomicity（原子性）：全部実行するか、全く実行しないか。
  - **C**onsistency（一貫性）：データの整合性が保たれる。
  - **I**solation（独立性）：同時に動くトランザクションどうしが互いに影響しない。
  - **D**urability（耐久性）：確定した結果は、障害が起きても消えない。
- **DDoS攻撃**（Distributed Denial of Service、分散型サービス妨害攻撃）：多数のコンピュータから一斉に大量の通信を送り、サーバを止める攻撃です。
- **MTBF**（Mean Time Between Failures、平均故障間隔）：故障してから次に故障するまでの平均時間です。長いほど壊れにくいことを表します。
- **MTTR**（Mean Time To Repair、平均修理時間）：故障してから直るまでの平均時間です。短いほど早く直ることを表します。

CAP定理・BASE特性・NoSQL

① 前提となる用語

- **分散システム**：データを複数のコンピュータ（ノード）に分けて置き、ネットワークでつないで1つのシステムとして動かすものです。
- **NoSQL**（Not only SQL）：表（リレーショナルデータベース）とSQL（Structured Query Language、データベースを操作する言語）を使わない、またはそれだけに頼らないデータベースです。大量で形式がばらばらなデータ（ビッグデータ）を、多数のノードに分散して扱うのに向いています。

② CAP定理

分散システムでは、次の3つを**同時にすべて満たすことはできない**、という定理です。

文字	英語	意味
C	Consistency（一貫性・整合性）	どのノードに聞いても、同じ最新のデータが返る
A	Availability（可用性）	一部が故障していても、必ず応答が返る
P	Partition tolerance（分断耐性）	ノード間のネットワークが切れても、システムが動き続ける

- **AとPを持つシステム**：可用性と分断耐性は満たしますが、**整合性は満たしません**。ネットワークが切れても応答を返すため、一時的に古いデータを返すことがあります。
- 見分け方：問題文に出てくる2文字以外の残り1文字が「満たさない」性質になります。AとPなら、満たさないのはCです。

③ BASE特性：NoSQLの考え方

- **B**asically **A**vailable（基本的に利用可能）：いつでも応答を返すことを優先します。
- **S**oft state（状態は厳密でない）：データが一時的に食い違っていてもよい、とします。
- **E**ventually consistent（**結果整合性**）：更新は、障害がなければ**いつかは全ノードに反映される**、という考え方です。
- 正解：「更新結果は、障害がなければ、**いつかは**全てのノードに反映される」。
- ひっかけの選択肢（どれもBASE（Basically Available・Soft state・Eventually consistent）ではありません）
  - 「2相コミットで全ノードに反映する」：2相コミットは、全ノードで同時に確定させる方式です。これはACID（Atomicity 原子性・Consistency 一貫性・Isolation 独立性・Durability 耐久性）側の、厳密な一貫性の考え方です。
  - 「全ノードが常に同じ結果を返す」：これは厳密な一貫性です。BASEは一時的な食い違いを許します。
  - 「ネットワークが分断されると受け付けない」：可用性を捨てている説明なので、BASEの「基本的に利用可能」と矛盾します。

④ NoSQLの4つの型

型	データの持ち方	例
キーバリュー型	1つのキーに値を対応付けて管理する。値の形式は自由	「user001」→ 任意のデータ
ドキュメント型	JSON（JavaScript Object Notation）などの文書を1件として管理する	{"name":"田中","age":35}
ワイドカラム型（列指向型）	**行ごとに列の数が自由**で、列ごとに複数の値を持てる	IoT（Internet of Things、モノのインターネット）機器の大量のデータ
グラフ型	ノード（点）とエッジ（線）で、ものどうしの関係を表す	SNS（Social Networking Service）の友達関係

NoSQLでない選択肢

- オブジェクト関係データベース：表をオブジェクト指向に拡張したもの。
- データウェアハウス：意思決定を支援するため、過去のデータも蓄積するもの。
- データディクショナリ：データの名前や形式などのメタ情報（データについての情報）を管理するもの。

インデックス（B+木・ハッシュ）＋3層スキーマ

① インデックスとは

- 表の検索を速くするための索引です。本の巻末にある索引と同じで、「この値はどの行にあるか」を記録しておきます。
- 注意点：インデックスを増やすと検索は速くなります。ただし、データを追加・更新・削除するたびに索引も更新するので、**更新の処理は遅くなります**。

② 2種類のインデックスの違い

	B+木インデックス	ハッシュインデックス
仕組み	値を**大小の順に並べた**木構造	値をハッシュ関数で計算し、格納場所を直接求める
得意な検索	**範囲検索**（○以上、今月分）、**前方一致**（'DB（Database、データベース）'で始まる）、並べ替え	**一致検索**（＝ ○○）だけ
苦手な検索	—	範囲検索・前方一致はできない（順序の情報を持たないため）

- **ハッシュ関数**：値から、決まった計算で格納場所の番号を作る関数です。例：社員番号を100で割った余りを、格納場所の番号にする。同じ値なら必ず同じ場所になるので、一致検索は一発で見つかります。一方で、値の大小の順序は失われます。

③ の解き方

問題は「B+木よりハッシュインデックスが適しているのはどれか」です。

選択肢	検索の種類	向いているインデックス
売上金額が1万円**以上**	範囲検索	B+木
売上年月日が**今月**	範囲検索	B+木
商品名が'DB'で**始まる**	前方一致	B+木
利用者ID（Identification、識別子）が**'1001'**（＝）	**一致検索**	**ハッシュ（正解）**

見分け方：「＝ ○○」の一致検索だけがハッシュ向きです。「以上」「今月」「で始まる」は、すべてB+木向きです。

④ 3層スキーマ

- **スキーマ**：データベースの構造の定義のことです。
- **ANSI/SPARC**（American National Standards Institute / Standards Planning And Requirements Committee）：米国の規格団体が定めた、データベースの構造を3つの層に分ける考え方です。

層	内容	例
外部スキーマ	利用者やプログラムから見た形	ビュー（必要な列だけを見せる仮想の表）
概念スキーマ	データ全体の論理的な構造	E-R（Entity-Relationship、実体関連）モデル、正規化、一意性制約・参照制約
内部スキーマ	ディスク上の**物理的な**格納方法	**インデックスの定義**、ファイルの構成

- 見分け方：「速さ」「物理的な格納」に関係するものは内部スキーマです。「データの意味や関係」に関係するものは概念スキーマです。

分散DB：2相コミット

おさらい（4/71より）

- **2相コミット**（two-phase commit）：複数のDB（データベース）にまたがる更新を「全部確定するか、全部取り消すか」にそろえる方法です。目的は**原子性**（Atomicity：全部実行するか、全く実行しないか）を守ることです。
- 登場人物は、**調停者**（全体のまとめ役。コーディネーターともいう）と**参加者**（各DB）です。

相	調停者 → 参加者	参加者 → 調停者
1相目	「確定（コミット）できますか？」と問い合わせる	「できます」「できません」と答える
2相目	全員OKならコミット、1人でもNGならロールバック（取消し）の**決定**を送る	完了したと返す

：参加者が「コミットか、ロールバックか」決められなくなる障害はどれか

- 正解：**調停者が、コミットかロールバックかの「決定」を参加者に送る直前に障害になった**とき。
- 理由：参加者はすでに「できます」と答えた後なので、自分の判断で取り消すことはできません。一方で、調停者からの決定も届かないので、確定することもできません。どちらにも動けず、**待ち続けるしかありません**（ブロック状態）。

ほかの選択肢（障害の起きた時点）	参加者が判断できる理由
問合せを送る直前	まだ何も約束していないので、取り消せばよい
参加者が「できる／できない」を返す直前	調停者は応答が来ないので、全員を取り消せばよい
参加者が「完了」を返す直前	決定はもう届いているので、それに従えばよい

ビュー

ビューとは

- SELECT文（データを取り出す命令）の結果に名前を付け、**仮想の表**として扱えるようにしたものです。**ビュー自体はデータを持たず**、見るたびに元の表から取り出します。そのため、元の表を更新すると、ビューにもそのまま反映されます。
- 目的：利用者に見せる列や行を絞る（セキュリティ）、複雑な問合せを簡単に使えるようにする。

の選択肢

選択肢	正誤	理由
ビューの列名を、元の表と**違う名前で**定義できる	○（**正解**）	例：CREATE VIEW V(氏名) AS（Autonomous System、ISPなどの組織単位のネットワーク） SELECT name … のように、別名を付けられる
列は抜き出せるが、**行は**抜き出せない	×	WHERE句（条件の指定）で行も絞れる
2つ以上の表を結合したビューは、**結合の仕方によらず更新できる**	×	結合したビューは、更新できないことが多い
和両立な表どうしの**和集合でビューを作れない**	×	UNION（和集合）でも作れる。和両立とは、列の数と型がそろっていること

更新できるビューの条件

- **1つの表**から作られていて、**元の行と1対1に対応する**ものです。
- 次のどれかを使ったビューは、元のどの行を直せばよいか決まらないので、更新できません。

- GROUP BY（グループごとに集計する）
- 集約関数（SUM 合計、AVG 平均、COUNT 件数など）
- DISTINCT（重複を除く）

正規化（第1〜第3正規形）

- 第1正規形＝繰返し項目がない（1つの欄に値が1つ）。
- 第2正規形＝第1正規形で、主キーの一部だけで決まる項目（部分関数従属）がない。
- 第3正規形＝第2正規形で、主キー以外の項目で決まる項目（推移的関数従属）がない。
- 第2→第3正規形にする手順：主キー以外の項目に従属する項目を、別の表に分ける。

外部キー・参照制約・CASCADE

- 外部キー＝ほかの表の主キーを参照する列。参照先にない値は入れられない（参照整合性）。
- ON DELETE CASCADE＝参照先の行を削除すると、それを参照している行も一緒に削除する。RESTRICT（NO ACTION）＝参照されている行は削除できない。SET NULL＝参照している列を NULL にする。

SQL（結合・GROUP BY・副問合せ）

- 内部結合（INNER JOIN）＝両方の表で一致する行だけ。左外部結合（LEFT OUTER JOIN）＝左の表の全行を残し、一致しない右側は NULL。
- GROUP BY で集計し、HAVING で集計結果を絞る（WHERE は集計前の行を絞る）。
- EXISTS＝副問合せの結果が1行でもあれば真。IN＝値が副問合せの結果の中にあれば真。
- 解き方：表を手で実際にたどり、結果の行を書き出す。

障害回復（ログ・チェックポイント・ロールバック／ロールフォワード）

- ロールバック（UNDO（更新前に戻す（取消し））＝更新前のログで、コミットしていないトランザクションの変更を取り消す。
- ロールフォワード（REDO（更新をやり直す））＝更新後のログで、コミット済みの変更をやり直す。
- チェックポイント＝バッファの内容をディスクに書き出す時点。システム障害後は、チェックポイント後にコミットしたものを REDO、障害時に未コミットのものを UNDO する。チェックポイント前にコミット済みのものは何もしない。
- 媒体障害（ディスク故障）＝バックアップから復元し、ログでロールフォワードする。

べき等・JSONデータ

- 表の設計から得られる情報を問う問題：必要な列（例：日付、代理店）がどの表にあり、キーで結合できるかをたどる。結合できる列がなければ、その情報は得られない。
- べき等（idempotent）＝同じ操作を何回行っても、結果が1回行ったときと同じになる性質（例：「値を5にする」はべき等、「値に1を足す」はべき等でない）。

データ管理者・データウェアハウス・マイニング

- DA（Data Administrator、データ管理者）＝全社のデータを管理する。論理設計、データ項目の標準化を行う。DBA（Database Administrator）＝DB の物理設計・性能・バックアップを行う。
- データウェアハウス＝分析用に、時系列のデータを統合して蓄積する DB。データマート＝特定の部門向けに切り出したもの。データレイク＝加工前の多様なデータをそのまま貯める。
- データマイニング＝大量のデータから、隠れた規則や相関を見つけ出す。

10 ネットワーク

TCPとUDP

① 2つの比較

どちらも、**OSI**（Open Systems Interconnection、開放型システム間相互接続）**基本参照モデルのトランスポート層（第4層）**プロトコル（通信の約束事）です。

	TCP（Transmission Control Protocol）	UDP（User Datagram Protocol）
接続	**コネクション型**：通信の前に相手と接続を確立する	**コネクションレス型**：接続せずにいきなり送る
信頼性	確認応答・再送・順序制御で、**確実に届ける**	確認応答も順序制御も**ない**
速さ	遅め	速い・軽い
使う場面	Webページ、メール、ファイル転送	時刻合わせ、名前解決、音声・動画

② ヘッダーの違い

ヘッダーとは、データの先頭に付ける制御用の情報です。

- **UDPのヘッダー**：送信元ポート番号、宛先ポート番号、長さ、チェックサムの4つだけです。
- **TCPだけにあるもの**：**シーケンス番号**、確認応答番号（ACK番号）、フラグ（SYN・ACKなど）、ウィンドウサイズ。
  - **シーケンス番号**：データに振る通し番号です。届いたデータを正しい順番に並べ直し、抜けているデータを見つけるのに使います。
  - **チェックサム**：データが壊れていないかを確認する値です。TCPにもUDPにもあります。
- ポート番号：1台のコンピュータの中で、どのアプリケーション宛ての通信かを区別する番号です。TCPにもUDPにもあります。

③ スリーウェイハンドシェイク：TCPの接続を確立する3回のやり取り

1. 接続したい側から **SYN**（Synchronize、接続要求）
2. 相手から **SYN/ACK**（了解＋こちらからも接続要求）
3. 接続したい側から **ACK**（Acknowledgment、確認応答）

ポートスキャン（攻撃の前の偵察で、開いているポートを調べること）では、SYNを送ってSYN/ACKが返ってきたら、そのポートは**開いている**と判断します。

④ どちらを使うか

UDPを使う	TCPを使う
NTP（Network Time Protocol、時刻合わせ）← **正解**	FTP（File Transfer Protocol、ファイル転送）
DNS（Domain Name System）の問合せ	POP3（Post Office Protocol version 3、メールの受信）
DHCP（Dynamic Host Configuration Protocol、IP（Internet Protocol、インターネットの通信規約）アドレスの自動割当て）	TELNET（遠隔ログイン）
SNMP（Simple Network Management Protocol、機器の監視）	HTTP（HyperText Transfer Protocol、Web）
音声・動画の配信	SMTP（Simple Mail Transfer Protocol、メールの送信）

覚え方：少しの量を素早く問い合わせるもの（時刻・名前・アドレス・監視）と、遅れたら意味がないもの（音声・動画）はUDPです。

⑤ の、ほかの選択肢

- 「コネクションレス」「確認応答や順序制御がない」とあれば、答えは**UDP**です。
- ICMP（Internet Control Message Protocol）：ネットワーク層（第3層）で、エラーを通知するプロトコルです。pingで使います。
- PPP（Point-to-Point Protocol）：データリンク層（第2層）で、2点間をつなぐプロトコルです。
- HTTP・SMTP：アプリケーション層（第7層）のプロトコルです。

cookie・リバースプロキシ・ダークネット

① cookie（クッキー）

- **cookie**：Webサイトが、利用者のブラウザに保存させる小さなデータです。ログイン状態や、買い物かごの中身などを覚えておくのに使います。

cookieの属性

属性	効果
Secure属性	**URL（Uniform Resource Locator、Webの住所）が https（暗号化通信）のときだけ、cookieを送る**（正解）。盗聴を防ぐ
HttpOnly属性	**JavaScript**（ブラウザ上で動くプログラム言語）**からcookieを読めなくする**。XSS（Cross-Site Scripting、クロスサイトスクリプティング）で盗まれるのを防ぐ
Expires／Max-Age属性	有効期限を決める
Path属性	指定したパスの下にアクセスするときだけ送る

- 見分け方：Secure は **https**、HttpOnly は **JavaScriptを禁止**、と対で覚えます。

ファーストパーティとサードパーティ

- ファーストパーティcookie：見ているサイトそのものが発行するcookieです。
- **サードパーティcookie**：見ているサイトとは**別の会社**（広告会社など）が発行するcookieです。いろいろなサイトをまたいで、利用者の行動を追跡（トラッキング）するのに使われます。
- 正解：サードパーティcookieをブロックすると、**複数のサイトをまたいだ追跡（トラッキング）をされなくなる**。

② プロキシ（代理で中継するサーバ）

種類	中継する向き	目的
リバースプロキシ	**インターネット → 内部の公開Webサーバ**（外から中への通信を、Webサーバの代わりに受け取って中継する）	負荷の分散、暗号化処理をまとめて引き受ける、Webサーバを直接外にさらさない（**正解**）
フォワードプロキシ（ふつうのプロキシ）	**内部のPC（Personal Computer） → インターネット**（社員のPCの代わりに、外部へアクセスする）	アクセスの記録、有害サイトへの接続の遮断

- 覚え方：reverse＝「逆」。ふつうのプロキシ（内→外）の逆向き（外→内）なので、リバースプロキシです。
- ひっかけ
  - 「攻撃を検知する」：IDS（Intrusion Detection System、侵入検知システム）の説明です。
  - 「仮想的な専用ネットワーク」：VPN（Virtual Private Network）の説明です。
  - 「内部のPC → インターネットを中継」：フォワードプロキシの説明です。

③ ダークネット

- **ダークネット**：インターネット上で、**どの機器にも割り当てられていない、使われていないIP（Internet Protocol、インターネットの通信規約）アドレスの範囲**です。
- 普通の通信は届かないはずなので、そこに届くパケットの大半は、**攻撃者やマルウェアが攻撃の対象を探すために、手当たり次第に送ったもの**です（正解）。
- 使い道：ダークネットに届くパケットを観測すると、攻撃の動向が分かります。

サブネット・CIDR

① CIDR（Classless Inter-Domain Routing、サイダー）

- **定義**：IP（Internet Protocol、インターネットの通信規約）アドレスを割り当てるときに、**クラスA〜Cという固定の区切りにとらわれず**、ネットワーク部とホスト部の境目を**好きな位置で区切って**、アドレスを無駄なく割り当てる方式です（正解）。
  - ネットワーク部：どのネットワークかを表す部分です。
  - ホスト部：そのネットワークの中の、どの機器かを表す部分です。

- 書き方：192.168.1.0**/24** のように、「/」の後ろにネットワーク部のビット数を書きます。/24なら、先頭の24ビットがネットワーク部です。
- 覚え方：Classless＝「クラスなし」。クラスの区切りを使わない、という意味です。
- 昔の**クラス方式**：クラスAは先頭8ビット、クラスBは16ビット、クラスCは24ビットと、ネットワーク部の長さが決まっていました。そのため、アドレスに無駄が多く出ました。

ほかの選択肢

略語	正式名	何をするか
DHCP	Dynamic Host Configuration Protocol	機器にIPアドレスを**自動で配る**
DNS	Domain Name System	ドメイン名（例：www.example.jp）と**IPアドレスを対応付ける**
NAPT	Network Address Port Translation	社内のプライベートアドレスを、1つのグローバルアドレスに**変換して**、複数台でインターネットを使えるようにする

② 計算問題を解くときの知識（今回は覚えるだけ）

- **サブネットマスク**：ネットワーク部を1、ホスト部を0で表した値です。例：/24 なら 255.255.255.0 です。
- **接続できるホスト（機器）の数**＝ 2^(ホスト部のビット数) − 2 です。
  - 2を引くのは、ホスト部が全部0のアドレス（ネットワークアドレス）と、全部1のアドレス（ブロードキャストアドレス）は機器に使えないからです。
- **ロンゲストマッチ**：ルーターが経路表（ルーティングテーブル）から転送先を選ぶとき、宛先に当てはまる経路が複数あれば、**サブネットマスクが最も長い（最も細かく一致する）経路**を選びます。

CSMA/CD・スパニングツリー

① CSMA/CD（Carrier Sense Multiple Access with Collision Detection、搬送波感知多重アクセス／衝突検出）

- 1本の通信路を、複数の機器（ノード）が共有して使うときの、送信のルールです。
- 手順
  1. **Carrier Sense**（搬送波感知）：送る前に、通信路が使われていないかを確かめます。
  2. 使われていなければ送信します。
  3. **Collision Detection**（衝突検出）：ほかの機器と同時に送ってしまい、データがぶつかったら（衝突したら）、それを検出します。
  4. 衝突したら、**ランダムな時間だけ待ってから、もう一度送ります**（正解）。待ち時間をランダムにするのは、全員が同じ時間だけ待つと、また衝突してしまうからです。
- 覚え方：「確認してから送る。ぶつかったら、ばらばらに待って送り直す」。

ほかの選択肢の正体

選択肢の説明	方式
ノードに順位を付け、送信権を順番に渡していく	トークンパッシング（バス型）
ノードを輪状につなぎ、送信権を表す特別なフレーム（トークン）を回す	**トークンリング**
割り当てられた時間の区切り（タイムスロット）にだけ送る	TDMA（Time Division Multiple Access、時分割多元接続）

② スパニングツリープロトコル（STP：Spanning Tree Protocol）

- **レイヤー2スイッチ**：MAC アドレス（Media Access Control address、ネットワーク機器ごとに付いた固有の番号）を見て、データの転送先を決める機器です。
- 問題点：スイッチを複数の経路でつなぐと、データ（イーサネットフレーム）が輪の中を**ぐるぐる回り続けるループ**が起き、ネットワークが止まってしまいます。
- **STP**：一部のポートを**止めて、経路を木の形（輪のない形）にする**ことで、ループを防ぎます（正解）。止めておいた経路は、ほかの経路が壊れたときの予備として使います。

- 覚え方：spanning tree＝ネットワーク全体に広がる「木」。木には輪がないので、ループが起きません。

ほかの選択肢

略語	正式名	何をするか
IGMP	Internet Group Management Protocol	マルチキャスト（特定のグループの機器にまとめて送る通信）のグループを管理する
RIP	Routing Information Protocol	ルーター（ネットワークどうしをつなぐ機器）が経路を決める。経由するルーターの数が最も少ない経路を選ぶ
SIP	Session Initiation Protocol	IP（Internet Protocol、インターネットの通信規約） 電話で、相手を呼び出したり通話を始めたり終えたりする

③ MTU（Maximum Transmission Unit、最大転送単位）

- 1回で送れるデータの**最大の大きさ**です。イーサネットでは 1,500 バイトです。これより大きいデータは、分割して送ります。

PoE・PLC・LPWA・NFC

4つとも、**名前と「何を使ってつなぐか」を対で覚えれば解けます。**

略語	正式名	何を使うか・何をするか	例
PoE	Power over Ethernet	**LAN（Local Area Network、構内ネットワーク）ケーブルで、通信と一緒に電気も送る**	無線LANのアクセスポイント、IP（Internet Protocol、インターネットの通信規約）電話機、監視カメラ
PLC	Power Line Communication	**電力線（コンセントの電線）を通信回線として使う**	家のコンセントにつなぐだけでLANになる
LPWA	Low Power Wide Area	**少ない電力で、1つの基地局が広い範囲（数km〜数十km）をカバーする無線通信**。速度は遅い	多数のIoT（Internet of Things、モノのインターネット）センサー、水道やガスの検針
NFC	Near Field Communication	**十数cm以内の、ごく近い距離の無線通信**	交通系ICカード、スマートフォン決済

- 覚え方
  - PoE＝Power over Ethernet：イーサネット（LANケーブル）の上で電気を送る。
  - PLC＝Power Line：電力線で通信する。
  - LPWA＝Low Power（省電力）＋Wide Area（広い範囲）。
  - NFC＝Near Field（近い距離）。
- 注意：PLCには、**Programmable Logic Controller**（工場の機械を制御する装置）という別の意味もあります。問題文が「通信」なら電力線通信、「工場の機械の制御」なら制御装置です。

ひっかけの選択肢

LPWAの特徴

選択肢の説明	正体
GHz帯の近距離無線で、4K・8Kの映像などを高速に送る	ミリ波などの高速無線。LPWAは遅い
電力線を使う有線通信で、スマートメーターの検針に使う	PLC
同じ基板の上の回路どうしをつなぐ、有線のシリアル通信	I2C（Inter-Integrated Circuit）など

LANケーブルで給電

- UPS（Uninterruptible Power Supply、無停電電源装置）：停電したときに、しばらく電気を供給する装置です。
- USB（Universal Serial Bus）：機器をつなぐ規格です。給電もできますが、LANケーブルは使いません。

NFCの説明

- 正解：**ピアツーピア（機器どうしが1対1で直接）通信する機能を持つ**こと。例：スマートフォンどうしをかざして、データを交換できます。

- ひっかけ
  - 「接触式ICカードの通信方法」：NFCは非接触（かざすだけ）なので誤りです。
  - 「通信距離は最大10m程度」：NFCは十数cmなので誤りです。
  - 「タッチセンサーで位置を検出する」：タッチパネルの説明なので誤りです。

IPアドレスの種類（プライベート・マルチキャスト・IPv6）

- プライベートアドレス：10.0.0.0/8、172.16.0.0/12、192.168.0.0/16。インターネットでは使えない。
- クラスD（224.0.0.0〜239.255.255.255）＝マルチキャスト（特定のグループの複数の宛先に同時に送る）。例：ルーティングプロトコル、映像配信。
- ブロードキャスト＝同じネットワーク（サブネット）内の全機器に送る。ルーターを越えない。
- IPv6＝128ビット。16ビットずつ16進数で8つに区切りコロンで並べる。連続する0の区切りは「::」で1回だけ省略できる。

DHCP・NAPT・ARP・ICMP

- DHCP（Dynamic Host Configuration Protocol）＝IP（Internet Protocol、インターネットの通信規約）アドレスなどを自動で割り当てる。PC（Personal Computer） は最初ブロードキャストで DHCPサーバを探す。ルーターを越えるにはDHCPリレーエージェントが必要。
- NAPT（Network Address Port Translation）＝プライベートアドレスとポート番号の組を、1つのグローバルアドレスとポート番号の組に変換し、複数の PC が同時にインターネットを使えるようにする。
- ARP（Address Resolution Protocol）＝IPアドレスから MAC（Media Access Control、機器固有の番号）アドレスを調べる（ブロードキャストで問い合わせる）。
- ICMP（Internet Control Message Protocol）＝IP通信のエラーや制御の情報を通知する。ping は ICMP のエコー要求・応答で接続を確認する。

DNS・オープンリゾルバ

- DNS（Domain Name System）＝ドメイン名と IP（Internet Protocol、インターネットの通信規約）アドレスを対応付ける。
- フルサービスリゾルバ（キャッシュ DNSサーバ）＝利用者の問合せを受けて、ほかの DNSサーバに順に問い合わせる。権威 DNSサーバ＝自分のゾーンの情報を答える。
- オープンリゾルバ＝誰からの問合せにも答えてしまうリゾルバ。DNSリフレクション（アンプ）攻撃に悪用される：送信元を被害者に偽って問い合わせ、大きな応答を被害者に送りつける。
- 対策：リゾルバは内部からの問合せだけに答える。公開用の権威サーバとリゾルバを分ける。

ルーター冗長化・スイッチ・VLAN・STP

- VRRP（Virtual Router Redundancy Protocol）＝複数のルーターを1つの仮想ルーターに見せ、マスターが故障したらバックアップが引き継ぐ。
- レイヤー2スイッチ＝MAC（Media Access Control、機器固有の番号）アドレスを見て転送先のポートを決める。レイヤー3スイッチ＝IP（Internet Protocol、インターネットの通信規約）アドレスでルーティングもできる。
- VLAN（Virtual LAN（Local Area Network、構内ネットワーク））＝物理的な配線に関係なく、論理的に LAN を分ける。VLAN間の通信にはルーティング（L3スイッチ）が必要。
- STP（Spanning Tree Protocol）＝レイヤー2のループを防ぐため、一部のポートを止めて木構造にする。

OSI参照モデル・プロトコルの階層

- 7層：物理・データリンク・ネットワーク・トランスポート・セッション・プレゼンテーション・アプリケーション。
- データリンク層（第2層）：イーサネット、PPP（Point-to-Point Protocol、シリアル回線用）、PPPoE（PPP over Ethernet）。ネットワーク層（第3層）：IP（Internet Protocol、インターネットの通信規約）、ICMP（Internet Control Message Protocol、エラー通知）。トランスポート層（第4層）：TCP（Transmission Control Protocol）、UDP（User Datagram Protocol）。

VPN（IPsec・L2TP・TLS）

- IPsec（IP（Internet Protocol、インターネットの通信規約） Security）＝ネットワーク層（第3層）で暗号化する。L2TP（Layer 2 Tunneling Protocol）＝データリンク層（第2層）のトンネル（暗号化なし、IPsec と組み合わせる）。TLS（Transport Layer Security）＝トランスポート層とアプリケーション層の間で暗号化する。

SDN・OpenFlow

- SDN（Software-Defined Networking）＝ネットワーク機器の制御機能（経路の決定）と転送機能を分け、制御をソフトウェアで集中して行う。
- OpenFlow＝SDN の代表的なプロトコル。OpenFlowコントローラーが、OpenFlowスイッチにフローテーブル（転送ルール）を設定する。

アプリケーション層のプロトコルとポート番号

- FTP（File Transfer Protocol）＝制御用に21番、データ転送用に20番の2つのポートを使う。
- HTTP＝80番、HTTPS＝443番。URL（Uniform Resource Locator、Webの住所） に「:8080」とあれば8080番を使う。
- SMTP（Simple Mail Transfer Protocol）＝メール送信、25番（サブミッションポートは587番）。POP3（Post Office Protocol version 3）＝110番、IMAP4（Internet Message Access Protocol）＝143番。
- MIME（Multipurpose Internet Mail Extensions）＝メールに画像などのファイルを添付する仕組み。Base64 で符号化すると、データ量は約4/3倍になる（3バイト→4文字）。

モバイル通信・ハンドオーバー・ローミング

- ハンドオーバー＝移動中の端末の通信を、基地局を切り替えながら途切れずに続ける。
- ローミング＝契約している事業者のサービス範囲の外で、提携している別の事業者の設備を使って通信する。
- テザリング＝スマートフォンを中継器にして、PC（Personal Computer） などをインターネットにつなぐ。

ルーティングプロトコル・SIP

- RIP（Routing Information Protocol）＝経由するルーターの数（ホップ数）が最も少ない経路を選ぶ（距離ベクトル型）。
- OSPF（Open Shortest Path First）＝回線の速さなどから計算したコストの合計が最小の経路を選ぶ（リンクステート型）。
- BGP（Border Gateway Protocol）＝ISP（Internet Service Provider、接続事業者） などの組織（AS：Autonomous System）の間で経路を交換する。
- SIP（Session Initiation Protocol）＝IP（Internet Protocol、インターネットの通信規約）電話で、相手の呼出し・通話の開始・終了などの制御を行う。音声そのものは RTP（Real-time Transport Protocol）で運ぶ。

メールの中継（第三者中継）

- 第三者中継（オープンリレー）＝外部から外部へのメールを中継してしまうこと。スパムの踏み台になる。対策：社内からの送信、または社内宛てのメールだけを中継する。

IoTの通信（MQTT など）

- MQTT（Message Queuing Telemetry Transport）＝IoT（Internet of Things、モノのインターネット） 向けの軽量な通信プロトコル。パブリッシュ／サブスクライブ型で、ブローカーがメッセージを中継する。

11 セキュリティ

攻撃手法

サイドチャネル攻撃

- 暗号機器の処理時間・消費電力・電磁波など、外から測れる物理的な情報から、秘密鍵を推測する攻撃です。
- 例：IC（Integrated Circuit、集積回路）カードの消費電力の波形を測って鍵を割り出す（電力解析攻撃）。暗号処理にかかる時間の差から鍵を割り出す（タイミング攻撃）。
- 見分け方：選択肢に「消費電力」「処理時間」「電磁波」が出てきたらサイドチャネル攻撃です。

ファジング

- 問題を起こしそうなデータ（ファズ）を大量に自動で入力し、ソフトウェアの未知の脆弱性（異常終了など）を見つける**テスト手法**です。攻撃の名前ではありません。

ボットネットとC&Cサーバ

- ボットネット：マルウェアに感染して、遠隔操作されるコンピュータ（ボット）の集まりです。
- C&Cサーバ（Command and Control server、指令サーバ）：ボットに攻撃の指令を出し、盗んだ情報を受け取るサーバです。

サイバーキルチェーン

- 攻撃を7段階に分けたモデルで、順番は「偵察→武器化→配送→攻撃（エクスプロイト）→インストール→C&C→目的の実行」です。
- 偵察段階：攻撃する前に、標的の組織や社員の情報（メールアドレス、SNS（Social Networking Service）など）を集める段階です。

その他の用語

- エクスプロイトコード：脆弱性を実際に突いて攻撃するプログラムです。
- ディープフェイク：AI（Artificial Intelligence、人工知能）で本物そっくりの偽の映像や音声を作ることです。例：社長の声をまねて電話をかけ、送金を指示する。
- XSS（クロスサイトスクリプティング、Cross-Site Scripting）：Webページに悪意のあるスクリプトを埋め込み、閲覧者のブラウザで実行させる攻撃です。
- SQL（Structured Query Language、データベースを操作する言語）インジェクション：入力欄にSQLの一部を入れて、データベースを不正に操作する攻撃です。対策はプレースホルダ（入力値を値としてだけ扱う仕組み）です。
- OS（Operating System、基本ソフト）コマンドインジェクション：入力にOSのコマンドを紛れ込ませて実行させる攻撃です。対策は、シェルを起動できる言語機能を使わないことです。
- ディレクトリトラバーサル：「../」でファイルの場所をさかのぼり、公開していないファイルを読む攻撃です。

可用性が出てくる4つの場面

場面	何の中の「可用性」か	意味
① 情報セキュリティの3要素	**C**onfidentiality（機密性）・**I**ntegrity（完全性）・**A**vailability（可用性）の頭文字で CIA（Confidentiality・Integrity・Availability、機密性・完全性・可用性）	必要なときに使えること。例：DDoS（Distributed Denial of Service、分散型サービス妨害攻撃）攻撃でサイトが止まると、可用性が失われます。
② CAP（Consistency・Availability・Partition tolerance）定理（分散データベース）	**C**onsistency（一貫性）・**A**vailability（可用性）・**P**artition tolerance（分断耐性）	一部が故障しても必ず応答を返すこと。分散システムでは、この3つを同時に満たせません。
③ BASE（Basically Available・Soft state・Eventually consistent）特性（NoSQL（Not only SQL、表形式に限らないデータベース)）	**B**asically **A**vailable（基本的に利用可能）・**S**oft state・**E**ventually consistent（結果整合性）	常に使えることを優先し、一貫性は最終的にそろえば良いとする考え方です。
④ システムの信頼性・サービスマネジメント	稼働率、サービス可用性、可用性管理	稼働率＝MTBF（Mean Time Between Failures、平均故障間隔）÷(MTBF＋MTTR（Mean Time To Repair、平均修理時間))。可用性管理は、サービスが合意どおりに使えるよう保証する活動です。

ACID（Atomicity・Consistency・Isolation・Durability、トランザクションの4性質）特性には可用性は入っていません。 ACIDはトランザクション（1回の処理のまとまり）の正しさの性質だからです。

ひっかけの見分け方

- 「ACIDに含まれないものは？」と聞かれたら、答えは可用性です。
- 「CIAやCAPに含まれるものは？」と聞かれたら、可用性も含まれます。

PKI・デジタル証明書・CRL・OCSP

① 仕組みの全体像

- **PKI**（Public Key Infrastructure、公開鍵基盤）：「この公開鍵は確かにこの会社のものだ」と、第三者が保証する仕組み全体のことです。
- **認証局**（CA：Certificate Authority）：その保証をする機関です。デジタル証明書を発行します。
- **デジタル証明書**：「持ち主の名前＋公開鍵＋有効期限＋シリアル番号」に、認証局が署名したものです。
  - サーバ証明書：サーバが本物であることを示します。
  - クライアント証明書：利用者のPC（Personal Computer）が、会社が許可した正規の端末であることを示します。

② 失効：有効期限の前に、証明書を取り消すこと

- 失効させる場面は、**秘密鍵が漏えいした**ときや、組織名が変わったときなどです。
- 期限切れは失効ではありません。期限が切れた証明書は、もともと使えないからです。

③ CRL（Certificate Revocation List、証明書失効リスト）

- 認証局が定期的に発行する、**有効期限内なのに失効させた証明書のシリアル番号の一覧**です。
- 正解の文：「有効期限内の証明書が失効したら、そのシリアル番号をCRLに載せる」。
- 間違いの選択肢
  - 「全証明書の有効期限を載せる」：載せるのは失効した証明書だけです。
  - 「失効後1年間載せるよう義務付け」：そのような規定はありません。
  - 「認証局の公開鍵がブラウザにあればCRLを見なくてよい」：公開鍵があっても、失効しているかどうかは分からないので、CRLの確認は必要です。

④ OCSP（Online Certificate Status Protocol）

- 証明書が失効しているかどうかを、**その場で（オンラインで）問い合わせる**通信規約です。問い合わせ先をOCSPレスポンダといいます。
- CRLは一覧をまとめてダウンロードして調べる方式、OCSPは1枚ずつその場で聞く方式、という違いです。
- ほかの選択肢
  - CHAP（Challenge Handshake Authentication Protocol）：チャレンジレスポンス方式のパスワード認証です。
  - LDAP（Lightweight Directory Access Protocol）：利用者情報などのディレクトリを検索する規約です。
  - SNMP（Simple Network Management Protocol）：ネットワーク機器を監視する規約です。
- ひっかけ：「秘密鍵の再発行の進み具合を問い合わせる」「期限切れの証明書の更新の進み具合を確認する」は、どちらも違います。OCSPで問い合わせるのは、失効しているかどうかだけです。

デジタル署名・コード署名

① デジタル署名の仕組み

- **目的**：①送信者が本人であること（なりすましでない）と、②途中で改ざんされていないこと、の2つを確認することです。**内容を隠す（暗号化する）ためのものではありません。**
- 手順
  1. 送信者は、メッセージからハッシュ値を作ります。ハッシュ値とは、メッセージを要約した短い値で、メッセージダイジェストともいいます。
  2. 送信者は、そのハッシュ値を**送信者の秘密鍵**（署名鍵）で暗号化します。これがデジタル署名です。署名をメッセージに付けて送ります。
  3. 受信者は、署名を**送信者の公開鍵**（検証鍵）で復号し、ハッシュ値を取り出します。
  4. 受信者は、届いたメッセージから自分でハッシュ値を作ります。3の値と一致すれば、本人からのもので、改ざんもないと分かります。

鍵の使い分け（最頻出）

	作る・暗号化する鍵	確かめる・復号する鍵
デジタル署名	**送信者の秘密鍵**	**送信者の公開鍵**
暗号化（秘密の通信）	受信者の公開鍵	受信者の秘密鍵

- 覚え方：署名は「本人しか持っていない鍵＝送信者の秘密鍵」で作ります。暗号化は「受信者だけが開けられる鍵」で開けます。
- ひっかけ：「署名鍵で暗号化して、内容が関係者以外に分からないようにする」は誤りです。署名の目的は内容を隠すことではないからです。「改ざんされた部分を特定できる」も誤りです。改ざんがあったことは分かりますが、どこが改ざんされたかまでは分かりません。

② コードサイニング証明書

- ソフトウェアにデジタル署名を付けるための証明書です。インストールするときに、**ソフトウェアの発行元が本物か**と、**改ざんされていないか**を確かめられます。
- 見分け方：問題文に「**ソフトウェア**」「**発行元**」とあれば、コードサイニング証明書です。

証明書	何を証明するか
コードサイニング証明書	**ソフトウェアの発行元**（正解）
サーバ証明書	Webサイトのサーバが本物であること
EV SSL（Secure Sockets Layer、TLSの前身の暗号化通信）証明書（Extended Validation、拡張検証）	サーバ証明書のうち、運営組織の実在を厳しく審査したもの
クライアント証明書	利用者の端末や本人であること

③ 署名を使う別の仕組み

- **DKIM**（DomainKeys Identified Mail、メールサーバによる署名）：送信側のメールサーバがメールにデジタル署名を付け、受信側のメールサーバがその署名を検証します。これで送信元のドメインが本物かを確認します。公開鍵はDNS（Domain Name System）で公開します。
  - 似ているが違うもの：SPF（Sender Policy Framework）は、署名ではなく、送信元メールサーバの**IP（Internet Protocol、インターネットの通信規約）アドレスを照合**する方式です。
- **DNSSEC**（DNS Security Extensions、DNS応答への署名）：権威DNSサーバ（そのドメインの情報を持つ正式なサーバ）が、**秘密鍵でDNSの応答にデジタル署名**を付けます。問い合わせた側（DNSキャッシュサーバ）が、公開鍵で署名を検証します。これで、応答が正しいサーバからのもので、改ざんされていないことが分かります。
  - ひっかけ：「送信元ポート番号をランダムにする」は、キャッシュポイズニング（偽の応答をキャッシュさせる攻撃）への別の対策で、DNSSECではありません。「共通鍵暗号によるメッセージ認証」も誤りで、DNSSECは**公開鍵暗号**によるデジタル署名を使います。

セキュリティ組織（CSIRT・PSIRT・SOC・JPCERT/CC）

① 3つの組織の違い

組織	正式名	守る対象	例
CSIRT（シーサート）	Computer Security Incident Response Team	**自分の組織の中**システム・情報	社内の在庫管理システム、販売管理システム、従業員の個人情報を扱うシステムで起きたインシデントに対応する
PSIRT（ピーサート）	Product Security Incident Response Team	**自社が顧客に販売した製品・サービス**	販売した製品の脆弱性の受付、修正プログラムの作成、利用者への注意喚起
SOC（ソック）	Security Operation Center	ネットワークや機器を**24時間監視**する	ログを見て攻撃を検知・分析し、CSIRTに知らせる

- **インシデント**：情報漏えい、マルウェア感染、不正アクセスなど、セキュリティを脅かす出来事のことです。
- **脆弱性**：ソフトウェアや製品にある、攻撃に使われうる弱点のことです。
- 見分け方
  - 「**自社製品**脆弱性」「**顧客に販売した**製品」とあれば **PSIRT** です（P＝Product）。
  - 「**社内**システム」とあれば **CSIRT** です。

- ひっかけ：在庫管理・販売管理・従業員情報のシステムは、「製品」という言葉が入っていても**社内のシステム**です。対応するのはCSIRTです。
- WHOISデータベース：ドメイン名の登録者情報を調べるデータベースです。

② JPCERT/CC（JPCERTコーディネーションセンター、Japan Computer Emergency Response Team Coordination Center）

- 日本のインシデント対応の**調整役**組織です。国内外のCSIRTどうしをつなぎます。
- **CSIRTマテリアル**：JPCERT/CCが作成した、**組織内にCSIRTを作るのを支援する資料**です。
  - ほかの選択肢：ISMS（Information Security Management System、情報セキュリティマネジメントシステム）ユーザーズガイド、証拠保全ガイドライン、組織における内部不正防止ガイドライン（IPAが作成）。
- **CSIRTガイド**：CSIRTを6種類に分類しています。そのうち**コーディネーションセンター**は、インシデントへの対応の中で、**CSIRTどうしの情報連携や調整を行う**CSIRTで、サービスの対象は**ほかのCSIRT**です。
  - 同じ6分類の例：組織内CSIRT（自組織が対象）、PSIRT型の製品ベンダーチーム（自社製品の利用者が対象）、分析センター（インシデントの傾向分析やマルウェアの解析をする）。

③ JVN

- JPCERT/CCとIPA（Information-technology Promotion Agency、情報処理推進機構）が共同で運営する、**ソフトウェアの脆弱性の情報と、その対策の情報を提供する**ポータルサイトです（正解）。
- ひっかけ選択肢とその正体
  - 「脆弱性の汎用的な評価手法を確立する」：CVSS（Common Vulnerability Scoring System）のことです。
  - 「脆弱性のタイプを識別する基準を提供する」：CWE（Common Weakness Enumeration）のことです。

情報セキュリティの特性と JIS Q 27000 の用語

- **JIS（Japanese Industrial Standards、日本産業規格） Q 27000**：情報セキュリティマネジメントシステム（ISMS：Information Security Management System）の**用語を定めた日本の規格**です。
- 試験では、**規格の定義文そのものを読ませて、どの用語か選ばせる**形で出ます。定義文と用語を対で覚えます。

① 特性の定義文

- エンティティ：人・組織・機器・プログラムなど、情報を扱う主体のことです。

用語	英語	JIS Q 27000 の定義文	一言で
機密性	Confidentiality	**認可されていない**個人・エンティティ・プロセスに、情報を**使用させず、開示しない**特性	見せない
完全性	Integrity	正確さ、および完全さの特性	改ざんされていない
可用性	Availability	**認可された**エンティティが**要求したとき**に、アクセスと使用ができる特性	使いたいときに使える
真正性	Authenticity	エンティティが、**それが主張するとおりのもの**である特性	本物である
信頼性	Reliability	**意図する行動と結果とが一貫している**特性	期待どおりに動く
否認防止	Non-repudiation	起きた事象や行動と、それを起こしたエンティティとを証明できる能力	後で「やっていない」と言わせない
責任追跡性	Accountability	エンティティの行動を、そのエンティティまでたどれること	誰がやったか追える

見分け方

- 「認可されて**いない**者に…使わせない」→ 機密性。
- 「認可された者が…使える」→ 可用性。
- 「主張するとおり」→ 真正性。
- 「行動と結果が一貫」→ 信頼性。

cとdは、真正性・信頼性ではありません。cは可用性、dは機密性の定義です。

② リスクに関する用語

用語	定義
リスクレベル	**結果とその起こりやすさの組合せとして表した、リスクの大きさ**（正解）
脆弱性	脅威に付け込まれる可能性のある、資産や管理策の**弱点**
リスク基準	リスクの重大さを評価するときの**目安とする条件**

- 「リスク対応の優先順位」は、リスクレベルの定義ではありません。
- 覚え方：リスクレベル＝影響の大きさ × 起こりやすさ、で決まる「大きさ」です。

クラウドのセキュリティ（CSPM・CASB・SAML）

① CSPM と、まぎらわしい仲間

略語	正式名	何をするか
CSPM	Cloud Security Posture Management（クラウドのセキュリティ態勢管理）	クラウドの**設定を監視**し、設定の不備（情報漏えいにつながる設定ミスなど）を見つけて、**自動で直す**（**正解**）
CASB（キャスビー）	Cloud Access Security Broker	社員が使っている**SaaS**（Software as a Service：インターネット経由で使うソフト）**利用状況を見える化**し、ルール違反を見つけて接続を制限する
UTM	Unified Threat Management（統合脅威管理）	ファイアウォールやマルウェアの検知など、**複数のセキュリティ機能を1つにまとめた**もの

- 覚え方
  - CSPM の **Posture**＝「姿勢・態勢」。クラウドの**設定の状態**が正しいかを見張ります。
  - CASB の **Broker**＝「仲介者」。社員とクラウドの**間に立って**、使い方を見張ります。

② SAML（Security Assertion Markup Language、サムル）

- 社内で一度ログインすれば、クラウドサービスにも**もう一度ログインせずに使える**ようにする仕組みです。これを**シングルサインオン**（SSO：Single Sign-On）といいます。
- 流れ
  1. 利用者が、社内の**認証サーバ**にID（Identification、識別子）とパスワードでログインします。
  2. 認証サーバが「この人は本人だと確認しました」という**認証結果**（アサーション：証明書のようなもの）を作ります。
  3. その認証結果を、**利用者のPC（クライアント）経由でクラウドサービスに渡します**（正解）。
- ポイント：**IDとパスワードそのものは、クラウドサービスに渡りません**。渡るのは「本人確認が済んだ」という結果だけです。
- ひっかけ
  - 「クラウドがIDとパスワードを要求する」「認証サーバからクラウドにIDとパスワードを送る」：パスワードを外に出さないのがSAMLなので、誤りです。
  - 「パスワードと乱数を組み合わせてハッシュ値にして送る」：これは**チャレンジレスポンス認証**説明です。

デジタルフォレンジックス

① 定義

- インシデント（不正アクセスや情報漏えいなど）が**起きた後に**、パソコンやサーバに残った電子データを集めて調べ、**何が起きたかを明らかにし、裁判でも使える証拠として残す**ことです。
- forensics は「法医学・鑑識」という意味で、コンピュータの鑑識にあたります。
- 手順は **収集 → 検査 → 分析 → 報告** の4段階です。

② 見分け方：「事後に、痕跡を調べる」ことかどうか

フォレンジックスに当たるのは、**すでに起きたことの痕跡を調べる**行為です。

行為	フォレンジックスか	本当は何か
サーバのハードディスクを解析し、**削除されたログファイルを復元して、不正アクセスの痕跡を見つける**	○（**正解**）	事後の調査
ログを1か所に集め、**リアルタイムに**関連付けて分析し、不正アクセスを**検出する**	×	**SIEM**（Security Information and Event Management）による監視。今起きていることを見つける
メールの本文と添付ファイルを**暗号化して**、情報漏えいを**防ぐ**	×	予防のための対策
実行する前に、プログラムのハッシュ値を脅威情報と照らし合わせ、**マルウェアかどうかを検査する**	×	マルウェア対策（予防）

- 覚え方：「**防ぐ**」「**リアルタイムに検出**」は事前・最中の対策です。「**復元して痕跡を発見**」が事後の調査、つまりフォレンジックスです。

③ 証拠として扱うときのルール

- **元のデータ（原本）には手を加えません**。複製を作って、複製の方を調べます。
- 原本と複製が同じであることは、**ハッシュ値**（データから計算した、要約の値）が一致することで証明します。
- 誰がいつ証拠を扱ったかを記録して、途中で書き換えられていないことを示せるようにします。

メールのセキュリティ（SPF・DKIM・DMARC・OP25B・サブミッションポート）

① 送信元ドメインが本物かを確かめる3つの技術

略語	正式名	確かめ方	見分けるキーワード
SPF	Sender Policy Framework	送信元ドメインのDNS（Domain Name System：名前とIP（Internet Protocol、インターネットの通信規約）アドレスの対応表）に「**このドメインのメールを送ってよいサーバのIPアドレス**」を登録しておく。受信側は、実際にメールを送ってきたサーバの**IPアドレスと照らし合わせる**	「**IPアドレス**」で照合
DKIM	DomainKeys Identified Mail	送信側のメールサーバが、メールのヘッダーに**デジタル署名**を付ける。受信側は、DNSに公開された**公開鍵で署名を検証**する。なりすましと**改ざん**両方が分かる	「**デジタル署名**」「**公開鍵**」
DMARC	Domain-based Message Authentication, Reporting and Conformance	SPF・DKIMの確認に**失敗したメールをどう扱うか**（受け取る・隔離する・拒否する）を、送信側が**ポリシー（方針）としてDNSに登録**する。認証結果のレポートも受け取れる	「**失敗したときの処理方法**」「**ポリシー**」

- 覚え方：SPF＝**住所（IPアドレス）**で確かめる。DKIM＝**署名**で確かめる。DMARC＝**失敗したときのルール**を決める。
- 似た名前の **S/MIME**（Secure/Multipurpose Internet Mail Extensions）：送信者**本人**がメールに署名や暗号化をする技術です。DKIMは**メールサーバ**が署名します。

② 迷惑メール（スパム）の送信を防ぐ仕組み

- **OP25B**（Outbound Port 25 Blocking）：プロバイダー（ISP：Internet Service Provider）が、自分の網の中から**外部のメールサーバの25番ポートへ直接送る通信を遮断**します。ウイルスに感染したPC（Personal Computer）が、勝手に迷惑メールを大量に送るのを防ぎます。
  - 25番ポートは、メールを送る SMTP（Simple Mail Transfer Protocol）の通常のポートです。
- **サブミッションポート（587番）**：正規の利用者がメールを送るための**専用ポート**です。ここでは **SMTP-AUTH**（SMTP認証：ID（Identification、識別子）とパスワードで、送信者が正規の利用者かを確かめる仕組み）を使って、**送信者を認証してから**送信します。
- OP25B で25番を止め、正規の利用者は587番＋SMTP-AUTHで送る、という組合せで使います。
- 見分け方：「サブミッションポート（587）を導入する目的」とあれば、答えは **SMTP-AUTH で送信者を認証する**です。

    - POP（Post Office Protocol、メール受信） before SMTP（メールを受信して認証したら、しばらく送信も許す古い方式）は、サブミッションポートの目的ではありません。

リスクマネジメント（JIS Q 31000）

- リスクアセスメント＝リスク特定（見つけて記述する）→リスク分析（性質と大きさ＝起こりやすさ×影響を理解する）→リスク評価（基準と比べて対応が必要か決める）の3つの全体。
- リスク対応の4つ：回避（原因の活動をやめる）、低減（対策で起こりやすさや影響を下げる）、移転・共有（保険・外部委託）、保有・受容（何もしないで受け入れる）。
- リスク受容基準はリスクアセスメントの前に決めておく（後で決めるのは不適切）。

暗号方式（共通鍵・公開鍵・楕円曲線）

- 共通鍵暗号（AES：Advanced Encryption Standard など）＝暗号化と復号に同じ鍵を使う。速い。n人が互いに通信すると鍵は n(n−1)/2 個必要。
- 公開鍵暗号（RSA（考案者3人の頭文字。公開鍵暗号）：Rivest-Shamir-Adleman など）＝公開鍵で暗号化し、本人の秘密鍵で復号する。鍵は1人1組（公開鍵と秘密鍵）なので n人で 2n 個。
- 楕円曲線暗号＝楕円曲線上の離散対数問題の難しさを使う公開鍵暗号。RSA より短い鍵長で同じ強さ。
- ハイブリッド暗号＝データは共通鍵で暗号化し、その共通鍵を公開鍵で暗号化して送る（TLS（Transport Layer Security、通信の暗号化） が採用）。

認証方式（チャレンジレスポンス・SAML・多要素）

- チャレンジレスポンス認証＝サーバが毎回異なる乱数（チャレンジ）を送り、クライアントはパスワードとチャレンジから計算したハッシュ値（レスポンス）を返す。パスワードそのものがネットワークに流れないので、盗聴されても再利用できない。
- SAML（Security Assertion Markup Language）＝認証の結果（アサーション）を XML（eXtensible Markup Language、タグでデータを記述する言語） でやり取りし、異なるサービスの間でシングルサインオン（SSO（Single Sign-On、一度の認証で複数サービス）、1回の認証で複数のサービスを使う）を実現する。
- 多要素認証＝知識（パスワード）・所持（スマホ、IC（Integrated Circuit、集積回路）カード）・生体（指紋）のうち、2種類以上を組み合わせる。
- 3Dセキュア 2.0（EMV 3-D Secure）＝ネットショッピングのクレジットカード決済の本人認証。リスクベース認証（普段と違う取引のときだけ追加認証する）を行う。

パスワード攻撃と対策（レインボー・リスト攻撃・ストレッチング）

- ブルートフォース攻撃（総当たり）＝あらゆる文字の組合せを試す。辞書攻撃＝よく使われる単語を試す。
- リバースブルートフォース＝パスワードを1つ固定して、利用者ID（Identification、識別子）を次々に変えて試す（アカウントロックを回避）。
- パスワードリスト攻撃＝ほかのサービスから漏れた ID とパスワードの組をそのまま試す（使い回しを狙う）。
- レインボーテーブル攻撃＝パスワードとハッシュ値の対応表（還元関数でつないで圧縮した表）を事前に作っておき、盗んだハッシュ値から元のパスワードを高速に探す。
- ソルト＝パスワードに利用者ごとの乱数を加えてからハッシュ化する（同じパスワードでもハッシュ値が変わり、事前計算した表が使えなくなる）。
- ストレッチング＝ハッシュ計算を何千回も繰り返し、1回の試行にかかる時間を延ばして、総当たりを遅くする。

WAF・IDS・IPS・ファイアウォール

- WAF（Web Application Firewall）＝HTTP（HyperText Transfer Protocol、Webの通信規約） の通信の中身を検査し、SQL（Structured Query Language、データベースを操作する言語）インジェクション・XSS（Cross-Site Scripting、悪意のスクリプトを実行させる攻撃） など Webアプリケーションへの攻撃を防ぐ。
- ファイアウォール＝IP（Internet Protocol、インターネットの通信規約）アドレスとポート番号で通信を許可・拒否する（中身は見ない）。

- IDS（Intrusion Detection System）＝不正な通信を検知して通知する。IPS（Intrusion Prevention System）＝検知して遮断もする。
- DMZ（DeMilitarized Zone）＝インターネットと社内 LAN（Local Area Network、構内ネットワーク） の間に置く緩衝区画。公開する Webサーバ・メールサーバ・DNS（Domain Name System、名前とIPアドレスの対応）サーバを置く。

DNSのセキュリティ（DNSSEC・キャッシュポイズニング）

- DNS（Domain Name System、名前とIPアドレスの対応）キャッシュポイズニング＝DNSキャッシュサーバに偽の応答を覚えさせ、利用者を偽サイトに誘導する。
- 対策：問合せのポート番号をランダムにする（送信元ポートランダム化）、DNSSEC を使う、外部からの再帰的な問合せに答えない。
- DNSSEC（DNS Security Extensions）＝DNS の応答にデジタル署名を付け、受信側が検証することで、応答の送信元が正しいこと（なりすましでないこと）と改ざんされていないことを確認する。暗号化はしない（機密性はない）。
- 公開用の DNS（権威サーバ）と社内用の DNS（キャッシュサーバ）は分ける。

セキュアOS・TPM・セキュアブート・耐タンパ

- セキュアOS＝強制アクセス制御（MAC（Media Access Control、機器固有の番号）：Mandatory Access Control）と最小特権で、管理者（root）であっても必要以上の権限を持たせない OS。不正侵入されても被害を局所化できる。
- TPM（Trusted Platform Module）＝PC（Personal Computer） のマザーボードに付けるセキュリティチップ。鍵の生成・保管、暗号処理を行い、耐タンパ性を持つ。
- セキュアブート＝PC の起動時に、OS やドライバのデジタル署名を検証し、改ざんされていないものだけを起動する。
- IC（Integrated Circuit、集積回路）カードの耐タンパ性を高める対策：内部の情報を読み出そうとすると、情報を消去する回路を組み込む。

ストレージの暗号化

- 自己暗号化ドライブ・ディスク全体の暗号化＝PC（Personal Computer） やディスクを盗まれても、中のデータを読めない。暗号化の鍵を TPM（Trusted Platform Module、セキュリティチップ） に保管し、ディスクだけ取り外しても復号できないようにする。

脆弱性の識別子・評価（CVE・CVSS・CWE）

- CVE（Common Vulnerabilities and Exposures）＝個々の脆弱性に付ける一意の識別番号（CVE-年-番号）。
- CVSS（Common Vulnerability Scoring System）＝脆弱性の深刻度を0〜10で評価する共通の方法。
- CWE（Common Weakness Enumeration）＝脆弱性の種類（型）を分類したもの。
- JVN（Japan Vulnerability Notes）＝JPCERT/CC（Japan Computer Emergency Response Team Coordination Center） と IPA（Information-technology Promotion Agency、情報処理推進機構） が共同で運営する、日本の脆弱性情報のポータルサイト。

政府・制度（ISMAP・CRYPTREC・電子署名）

- ISMAP（Information system Security Management and Assessment Program、政府情報システムのためのセキュリティ評価制度）＝政府が利用するクラウドサービスを、事前に評価して登録する制度。
- CRYPTREC（Cryptography Research and Evaluation Committees）＝電子政府で使う暗号の安全性を評価する。「電子政府推奨暗号リスト」を作成する（デジタル庁・総務省・経済産業省）。

ネットワークのセキュリティ（IPv6・特定IPからの通信）

- 送信元IP（Internet Protocol、インターネットの通信規約）アドレスで制限するだけでは、なりすまし（IPスプーフィング）を防げない。認証と組み合わせる。

ログ分析・攻撃の痕跡

- ログから攻撃を見つける：同じ送信元から短時間に大量のログイン失敗（総当たり）、URL（Uniform Resource Locator、Webの住所） に「../」（ディレクトリトラバーサル）、「' OR 1=1」（SQL（Structured Query Language、データベースを操作する言語）インジェクション）、「